

Chapitre 4 Machines à états finis

1. Partie opérative et partie contrôle
2. Graphes d'état
3. Transposition – Machine de Moore
 1. Codage des états
 1. Aléatoire
 2. Adjacent
 3. One Hot
 4. Total Synchrone
4. Machine de Mealy
5. Description VHDL

Objectifs

- Modéliser un système numérique complexe en VHDL
- Identifier les parties d'un système numérique complexe
- Représenter la partie contrôle sous forme d'un graphe d'état
- Traduire le diagramme d'état en VHDL
- Optimiser un diagramme d'état
- Passer du graphe d'état à la machine à état
- Donner la différence entre les différents types de MAE
- Réaliser la partie chemin de données/partie opérative en RTL

1 Partie Opérative / Contrôle

- Système Numérique Complexe

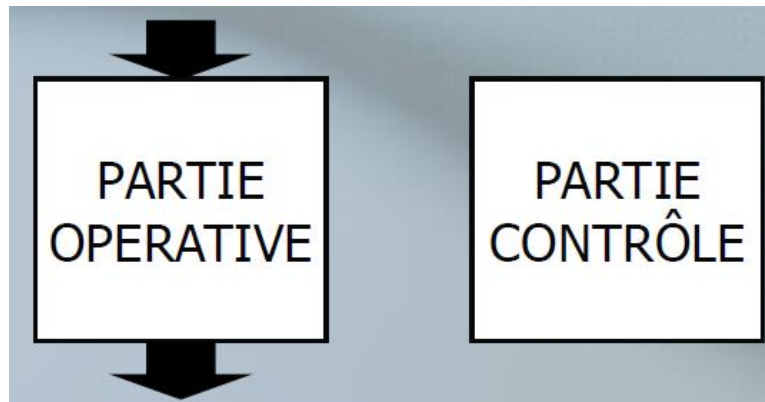
- Partition des Ressources

- PARTIE OPÉRATIVE : Calcul et Mémorisation

- Elle réalise les opérations sur les données en entrées et fournit des résultats qui sont exploités par la partie contrôle ou directement envoyés en sortie.

- Mémorisation, +, -, comparaison $<$, $>$, $=$, décalage, opération logique

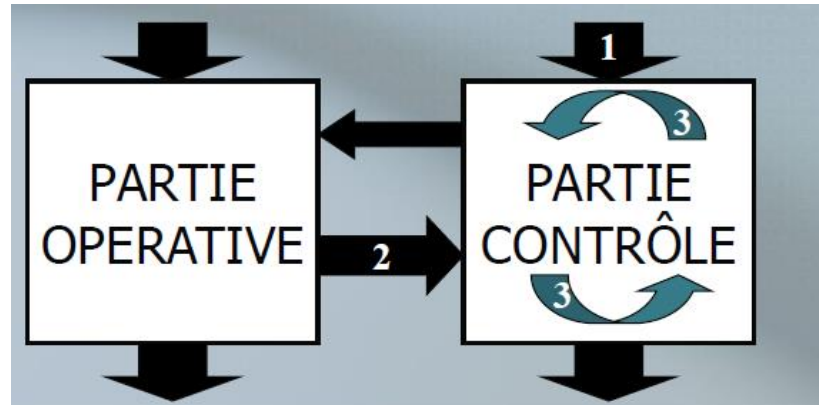
- PARTIE CONTRÔLE : Contrôle



Partie Opérative / Contrôle

P. Ctrl commande les actions de la P. Opérative en fonction

- 1) Des entrées du système
- 2) Des résultats engendrés par la partie opérative
- 3) De la propre évolution de la partie contrôle



Partie Contrôle

- Permet de déterminer le déroulement des états successifs du système numérique
 - État: est caractérisé par l'ensemble des valeurs des sorties à un instant donné
- Est Implémenté sous la forme d'une machine à états
 - Qui est la traduction électronique d'une représentation graphique de l'évolution du système
 - Est modélisée par un GRAPHE D'ÉTATS

Machines à états finis

- FSM : Définition Les Machines à états finis (FSM en anglais) sont utilisées pour décrire des comportements séquentiels liés au contrôle des parties opératives Cet aspect séquentiel fait intervenir la notion d'état interne implémentés dans les circuits sous forme de registres

Automate à états finis

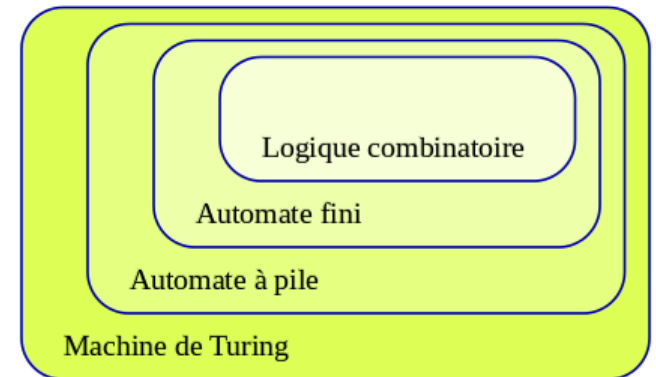
- Un automate fini est une construction mathématique abstraite, susceptible d'être dans un nombre fini d'états, mais étant un moment donné dans un seul état à la fois ;
- **l'état** dans lequel il se trouve alors est appelé l'« *état courant* ».
- Le passage d'un état à un autre est activé par un événement ou une condition ; ce passage est appelé **une « transition »**.
- Un automate particulier est défini par l'ensemble de ses états et l'ensemble de ses transitions.
- L'automate est représentée par un diagramme d'état-transition.

Automate à états finis

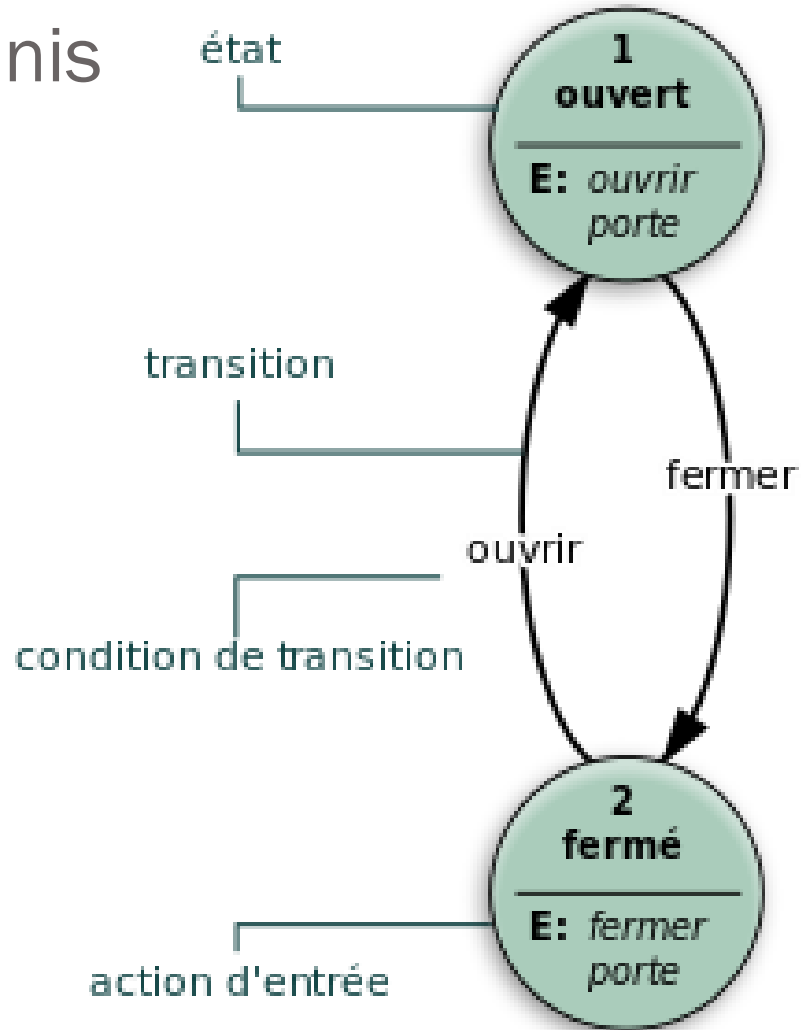
Hiérarchie d'automates

- Un automate fini avec un état unique est appelé parfois « combinatoire » ;
- Les automates finis peuvent être classés principalement en deux catégories, les **accepteurs** (leur sortie est soit une acceptation soit un rejet) et les **transducteurs** (*ce sont des automates finis avec une sortie*)
- Il existe de nombreuses **variantes** de **diagrammes d'états-transitions**

Hiérarchie d'automates

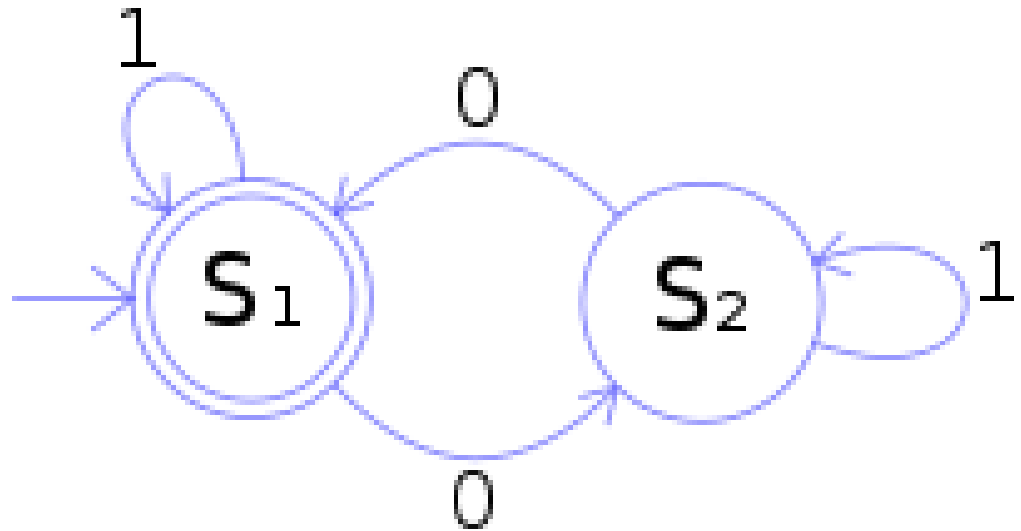


Automates Finis



Automates Finis

- Exemple

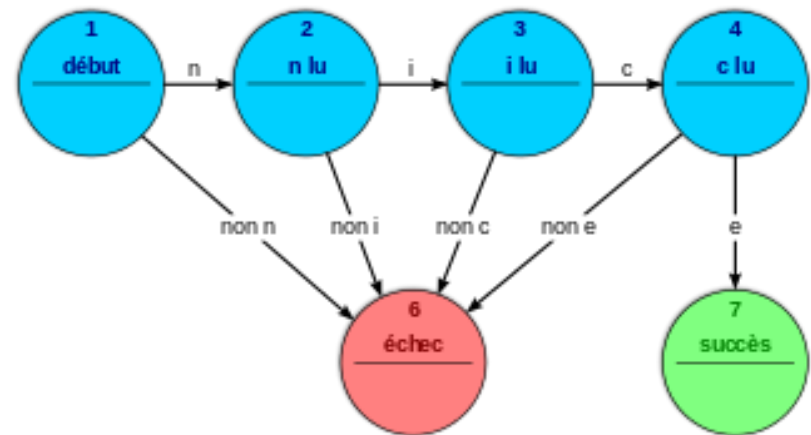


Représentation d'un automate fini qui détermine si une suite binaire contient un nombre pair de 0. L'état initial et l'état final est S1.

Classe d'automate finis

- **Accepteur**

- L'exemple de la figure montre un automate fini qui accepte le mot « nice ». Dans cet automate, seul l'état 7 est acceptant.

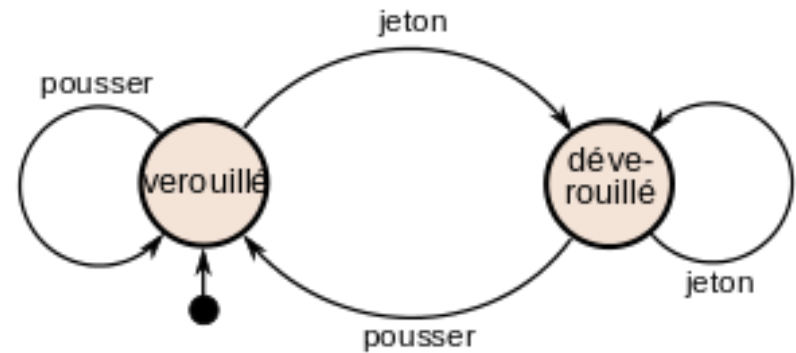


Automates Finis

• Transducteur

- Un exemple très simple d'un mécanisme que l'on peut modéliser par un automate fini est un portillon d'accès

Si l'on insère un jeton, c'est-à-dire si l'on effectue une « entrée » **jeton**, on passe de l'état **verrouillé** à l'état **déverrouillé**. Dans l'état **déverrouillé**, ajouter des jetons supplémentaires n'a pas d'effet, et ne change pas l'état. Mais dès qu'un usager tourne le bras du portillon, donc fournit un **pousser**, la machine retourne à l'état **verrouillé**.

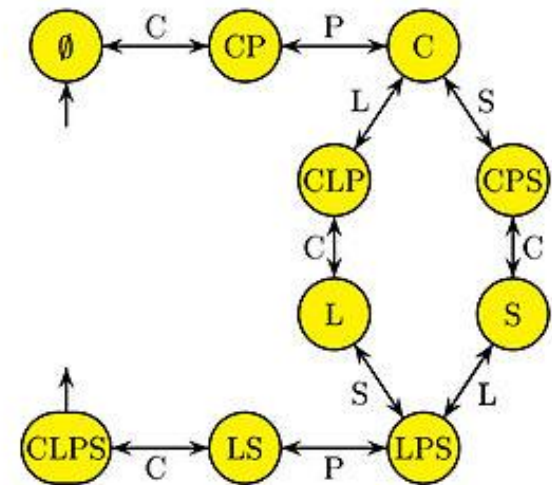


Automates Finis

un passeur qui doit faire traverser, d'une rive à l'autre, un loup, une chèvre et un chou. Sa barque ne lui permet que d'emporter un seul des trois objets à la fois et, bien entendu, il ne peut laisser ensemble loup et chèvre ni chèvre et chou.

Dans le diagramme ci-contre, un état représente ce que le passeur a déjà pu transporter sur l'autre rive (« P » représente le passeur, « L » le loup, « C » la chèvre, et le chou a été noté par « S ») : au début rien, à la fin tout.

- Quelle langue reconnaît-il?



Représentation d'une FSM

- On représente de manière équivalente une FSM Soit par un diagramme d'états
Soit par une table de vérité correspondant aux fonctions de transitions et de générations

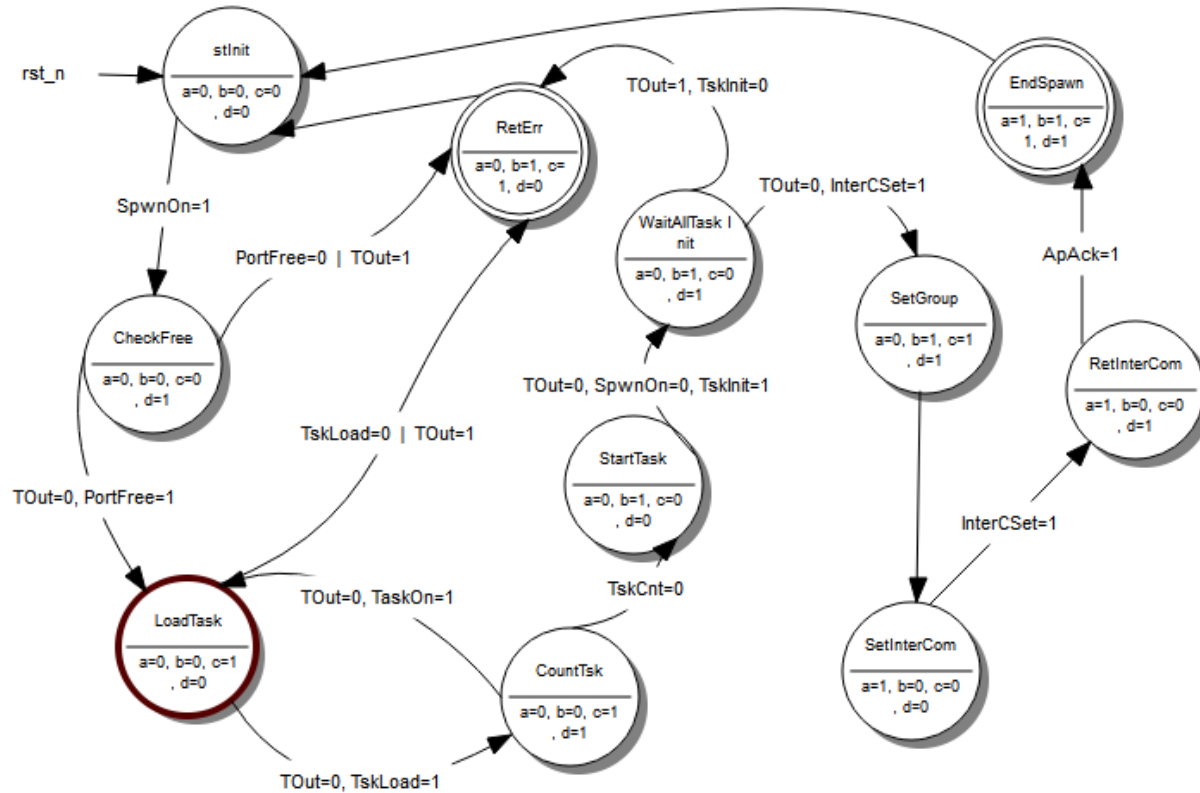
Modélisation des FSM

- Utilisation de registres d'états
 - Explicite : énumération des états
 - Implicite : dépend de la structure du modèle
- Introduction d'une méthode d'initialisation
 - Reset synchrone Ou
 - asynchrone
- Encodage des états
 - La façon de coder les états à un impact direct sur la complexité de la fonction de transition, donc sur les performances du circuit
- Choix Mealy / Moore
 - Moore, plus de maîtrise sur le comportement
 - Mealy, plus de flexibilité puisque les sorties peuvent changer au sein d'un même état
- Transitions
 - La transition d'un état à l'autre est synchronisée sur signal d'horloge implicite qui n'est représenté ni dans la tables d'états ni dans les diagrammes d'états

Types de machines

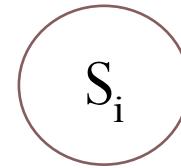
- Machine de Moore : Codage des sorties directement dans les états
- Machine de Mealy : Codage des sorties sur les transitions

2 Graphes d'état



Graphe d'états

- Description du comportement du système:
 - Son état



- Pour chaque état, va correspondre une série d'actions

Sortie 1 = ...
Sortie 2 = ...

- Son évolution
 - En fonction (ou non) d'événements sur les entrées du système



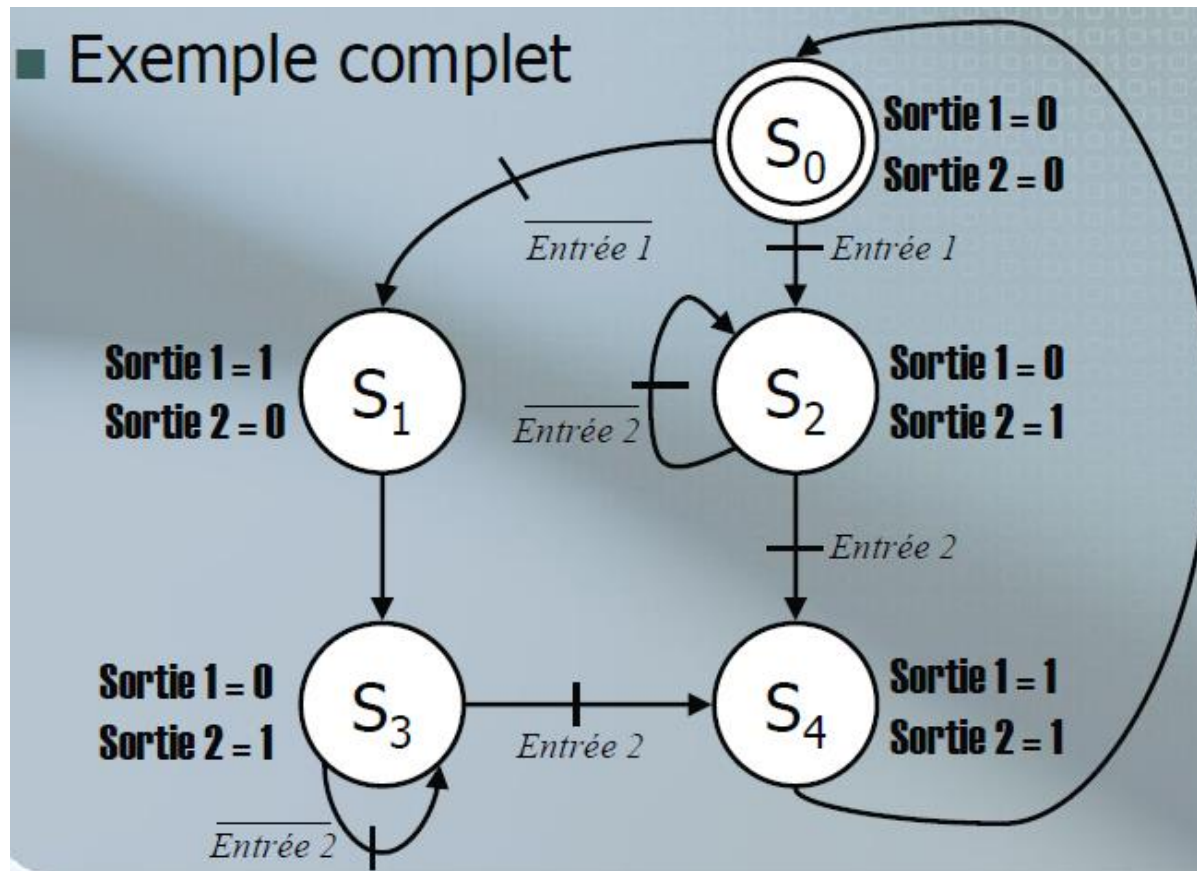
2.1 Évolution des États

- A chaque étape (cycle d'horloge), le système peut changer d'état
- Il existe deux types de transitions
 - Conditionnelle
 - La transition s'effectue si un événement survient sur les entrées
 - Inconditionnelle
 - La transition s'effectue indépendamment de tout événement sur les entrées



Graphe d'états

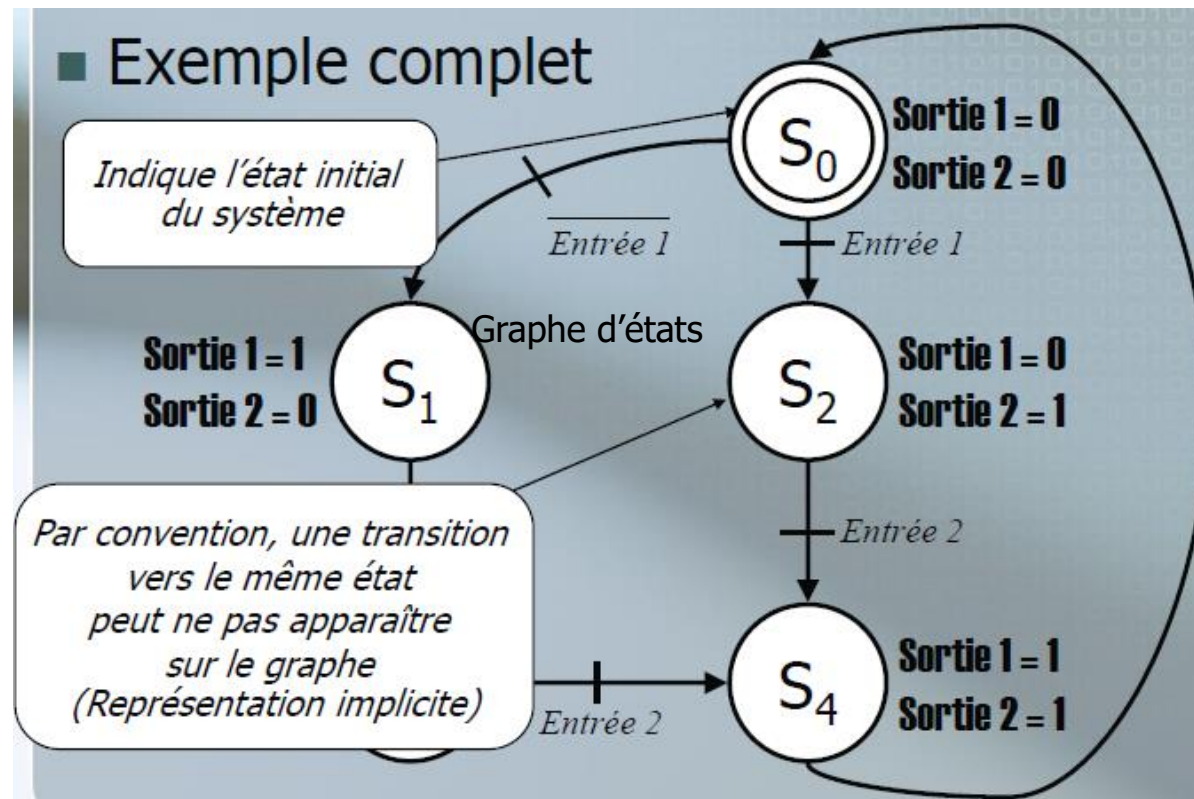
- Exemple complet



2.2 Transitions

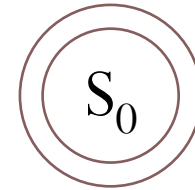
- Pour être synthétisable, une machine à états doit être déterministe
 - On ne peut pas se trouver simultanément dans plusieurs états
 - Il ne peut y avoir qu'une transition par état si celle-ci est inconditionnelle
 - Il ne peut pas y avoir qu'une seule transition conditionnelle pour un état: soit 0, soit 2 ou plus

Graphe d'états



Etat d'initialisation

- Dans le graphe, l'état d'initialisation représente l'état dans lequel se retrouve le système après un RESET ASYNCHRONE



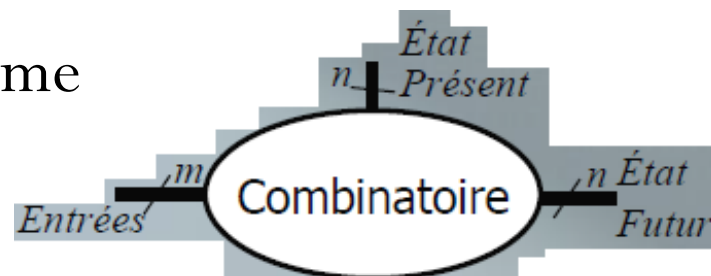
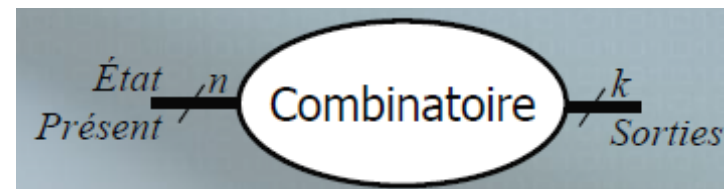
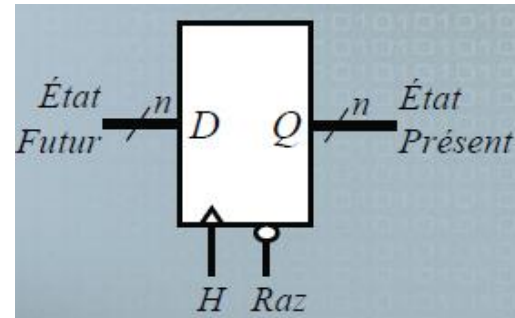
- Les transitions d'un état vers l'état initial par suite d'un RESET ASYNCHRONE ne sont pas explicitées dans le graphe
- SEULES les TRANSITIONS SYNCHRONES (sur un front d'horloge) sont représentées sur le graphe d'état

Réalisation Machine à États

- Transposition du graphe d'états en un système électronique
- Méthode de réalisation

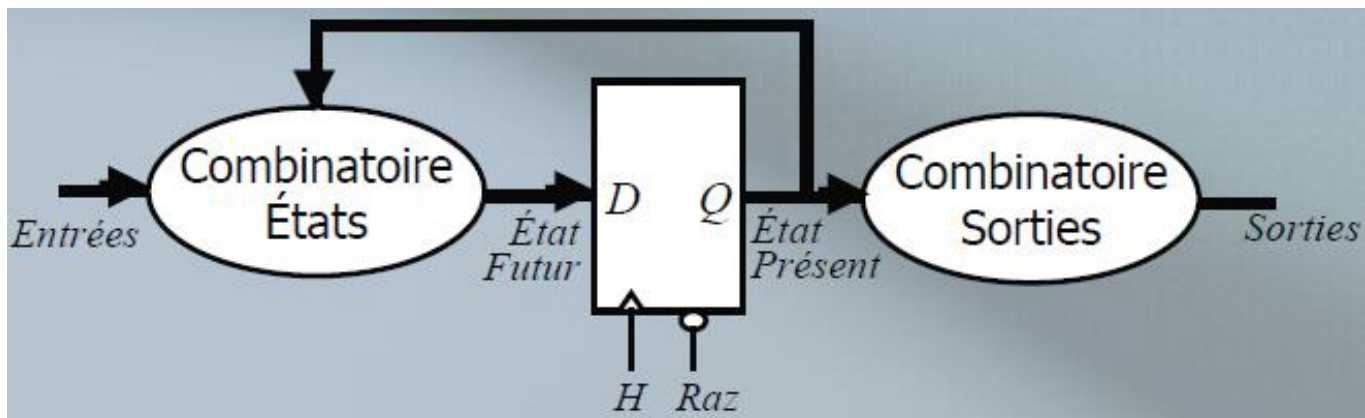
Transposition Électronique

- Éléments nécessaires
- État du système
 - Registre d'états
- Actions du système
- Évolution du système



Machine de Moore

- Horloge & RAZ sont des entrées implicites du système
- Elles n'apparaissent pas sur le graphe d'états
- Une RAZ asynchrone ramène à l'état initial



Réalisation Machine d'états

1. Spécifications du cahier des charges (*CRUCIAL*)
2. Identification des entrées/sorties de la MAE
3. Détermination des états (nombre, fonction)
4. Établissement du graphe d'états.

5. Détermination de l'encodage des états
6. A l'aide éventuellement d'une table de transition des états Calcul des équations de l'état futur et des sorties
7. Réalisation

Réalisation Machine d'états

1. Spécifications du cahier des charges (*CRUCIAL*)

2. Détermination des états (nombre, fonction)

3. Identification des entrées/sorties des états

4. Établissement du graphe d'états.
**Etapas 1 à 4 à effectuer
manuellement**

5. Détermination de l'encodage des états

A l'aide éventuellement d'une table de transition des états

6. Calcul des équations de l'état futur et des sorties

7. Réalisation
**Etapas 5 à 7 semi-automatique
(outils de synthèse)**

3.1 Codage des états

1. Binaire
2. Adjacent
3. One Hot
4. Total Synchrone
5. Aléatoire

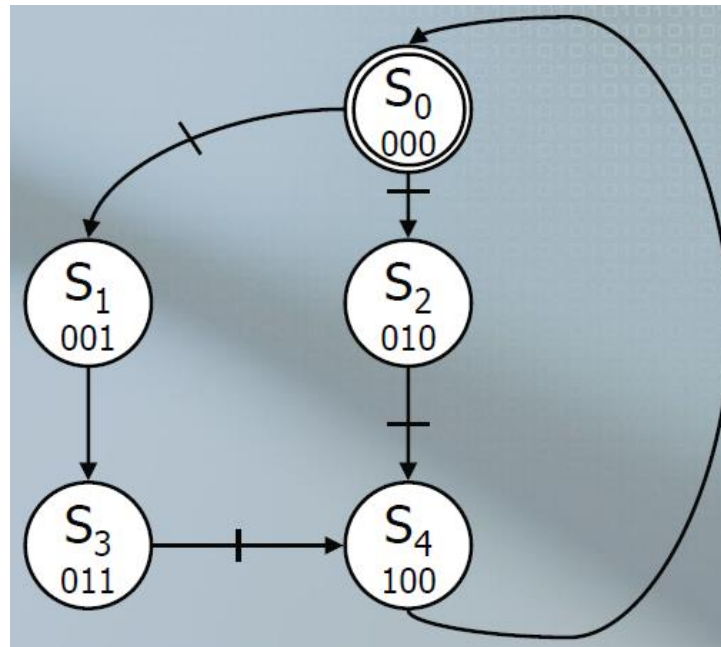
Codage des états

- L'état du système est représenté par la valeur stockée dans le registre d'états
- Détermination d'un code pour chaque état
- La longueur du code détermine la taille du registre d'états

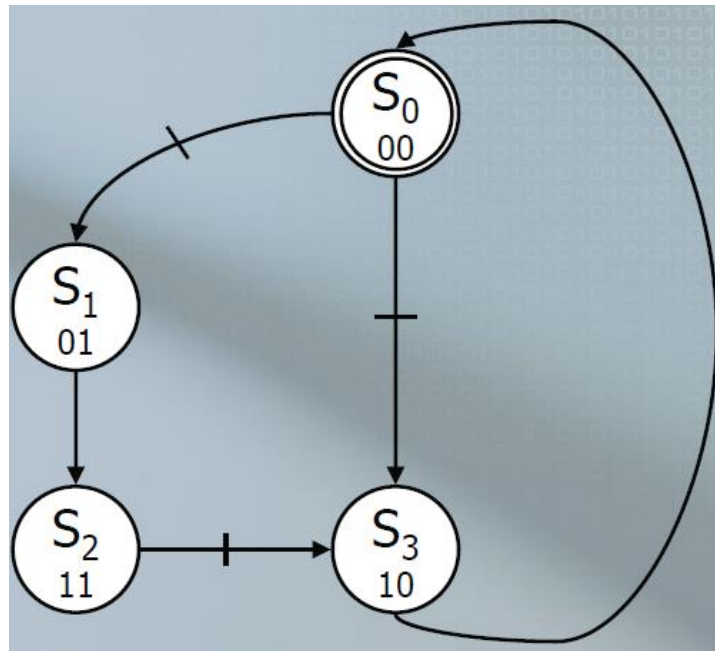
Codage des états

- **Codage binaire**
- Nombre de bascules minimales nécessaires
 - Si 2^n états dans le graphe
 - n bascules dans le registre d'états
 - Choix d'un code binaire pour chaque état
- Codage aléatoire
 - Le code de chaque état est choisi arbitrairement par le concepteur
- Codage adjacent
 - Un seul bit peut être modifié pour chaque transition
 - Cet encodage n'est pas toujours possible selon les graphes

Exemple Codage Binaire

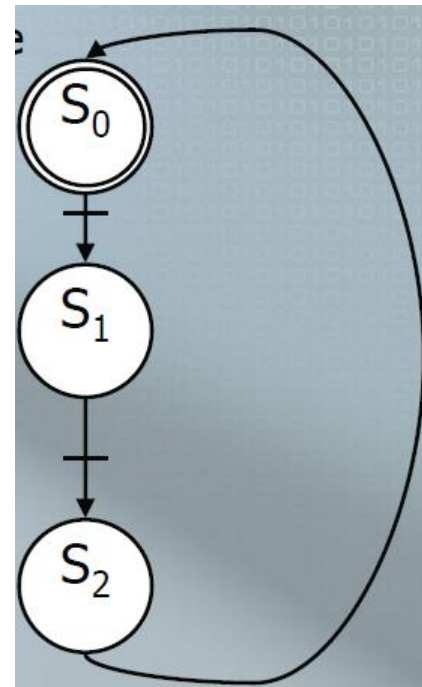


Exemple de codage adjacent



Exemple de codage adjacent

impossible sur ce graphe

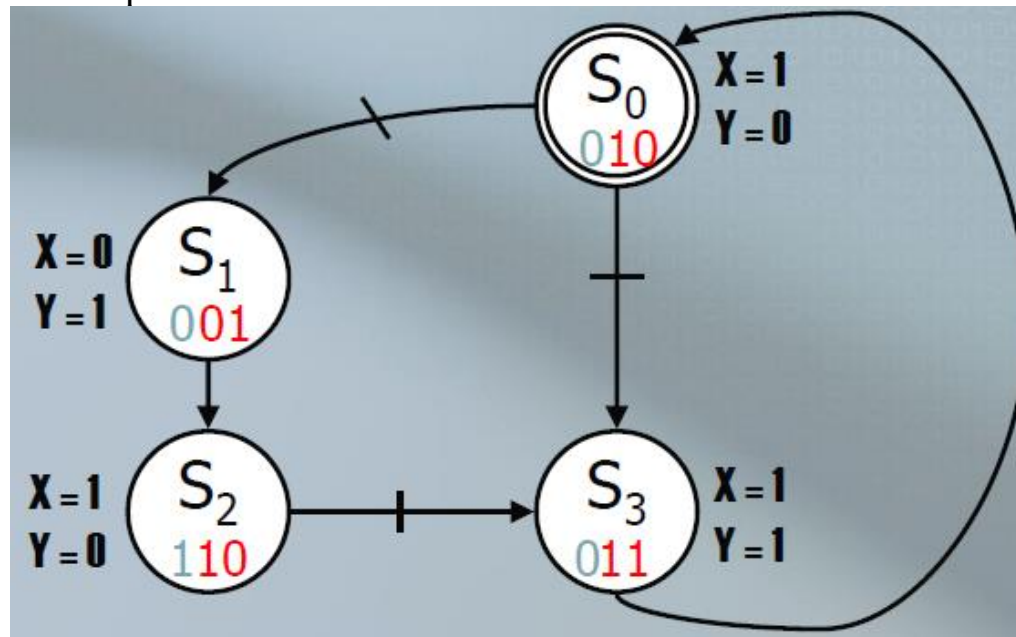


Autres codages

- Codage total synchrone
 - Principe: Coder les états de façon à éliminer le combinatoire de sortie
 - Peut nécessiter plus de bascules pour le registre

Codage Total Synchrone

- Méthode
 - Recopier l'état des sorties dans le codage des états
 - Compléter éventuellement avec d'autres bits

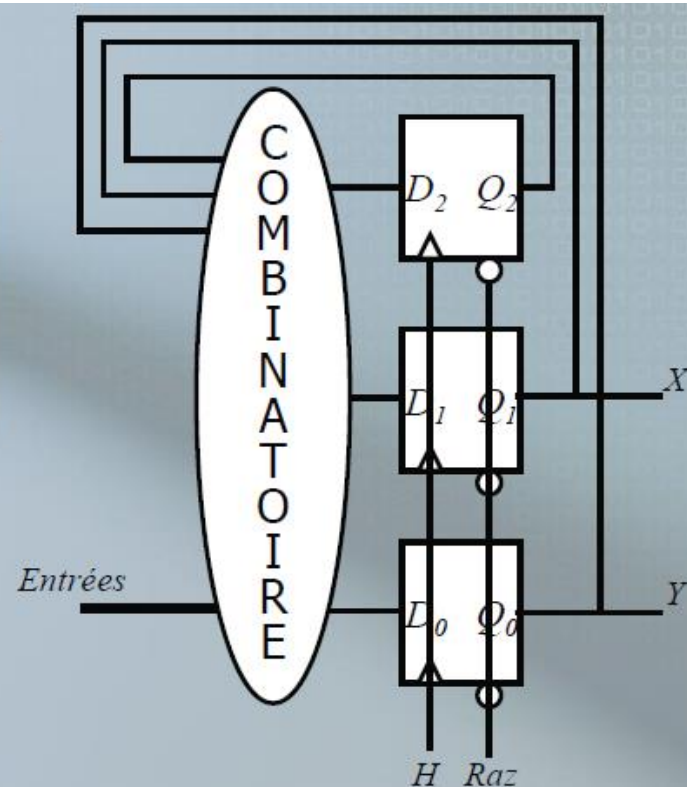


Codage Total Synchrone

Etat Pres.	Q_2	Q_1	Q_0	X	Y
S0	0	1	0	1	0
S1	0	0	1	0	1
S2	1	1	0	1	0
S3	0	1	1	1	1

$$X = Q_1$$

$$Y = Q_0$$



Autres codages

- Codage One-Hot
 - Principe: 1 bascule par état
 - 2 bascules ne peuvent être à '1' en même temps
 - Si $Q_i = 1$ alors la machine est à l'état i
- Permet de simplifier les combinatoires

Codage One-Hot

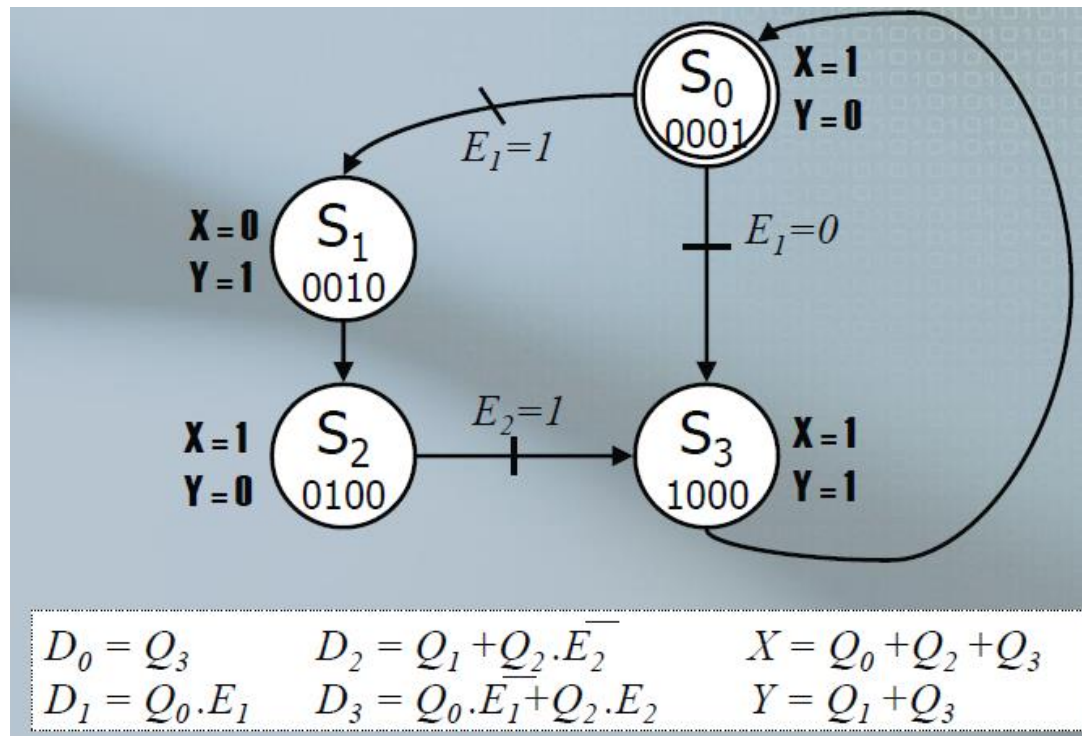


Table de Transition

- Table de transition du graphe du slide 202
- Les transitions s'effectuent sur des fronts d'horloge
- Une machine d'état est un système SYNCHRONE

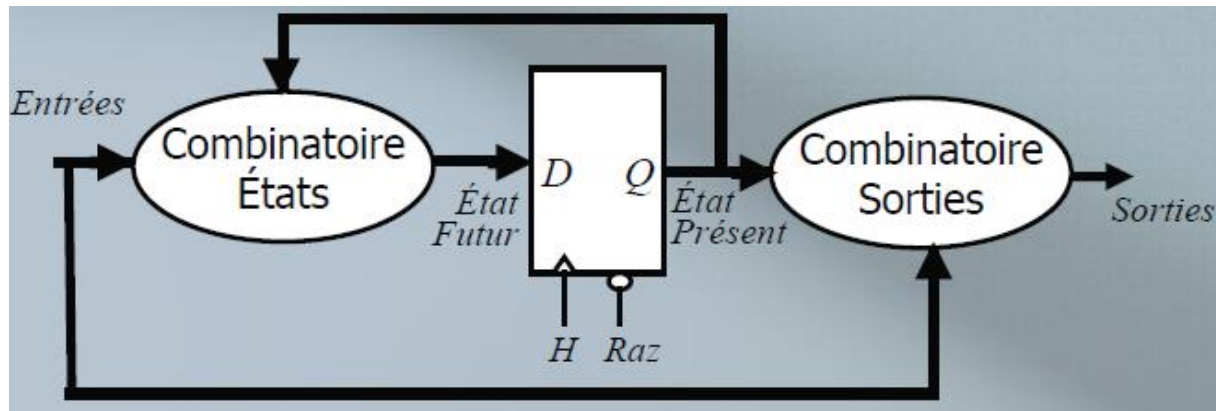
Etat Pres.	E1	E2	Etat Fut.	Sortie 1	Sortie 2
S0	0	0	S1	0	0
S0	0	1	S1	0	0
S0	1	0	S2	0	0
S0	1	1	S2	0	0
S1	0	0	S3	1	0
S1	0	1	S3	1	0
S1	1	0	S3	1	0
S1	1	1	S3	1	0
S2	0	0	S2	0	1
S2	0	1	S4	0	1
S2	1	0	S2	0	1
S2	1	1	S4	0	1
S3	0	0	S3	0	1
S3	0	1	S4	0	1
S3	1	0	S3	0	1
S3	1	1	S4	0	1
S4	0	0	S0	1	1
S4	0	1	S0	1	1
S4	1	0	S0	1	1
S4	1	1	S0	1	1

Transposition Matérielle

- Du graphe à la table des transitions
- De la table des transistions aux équations des portes logiques...

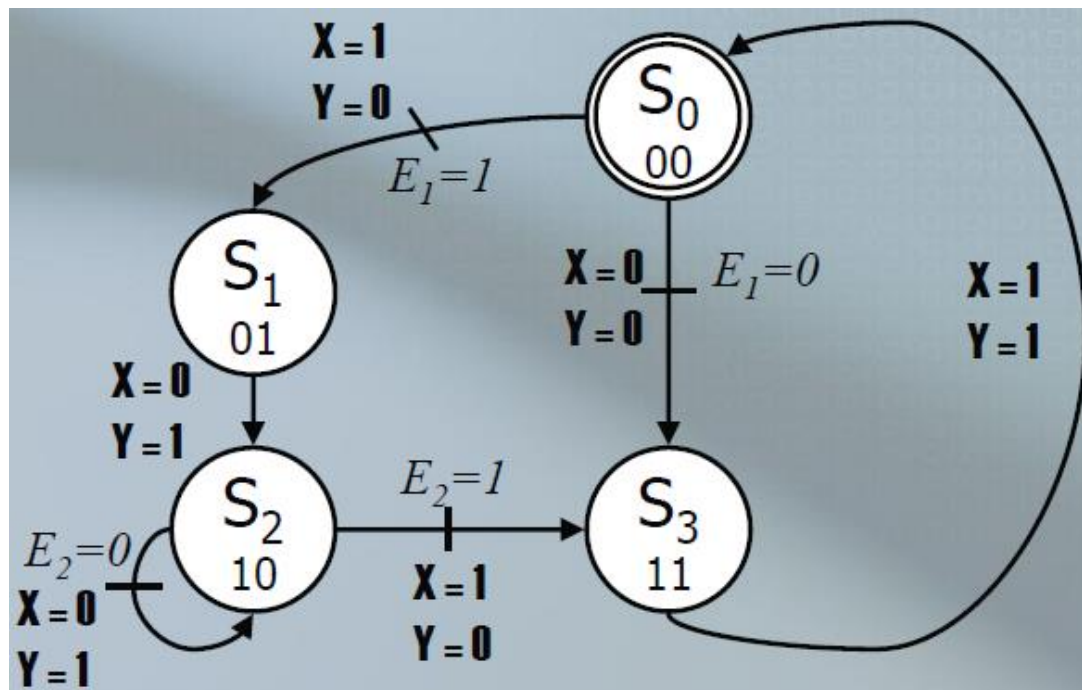
Machines de Mealy

- Machines de Mealy
- Structure similaire à Moore
- Différence: Le calcul des sorties dépend
 - De l'état présent
 - Des entrées



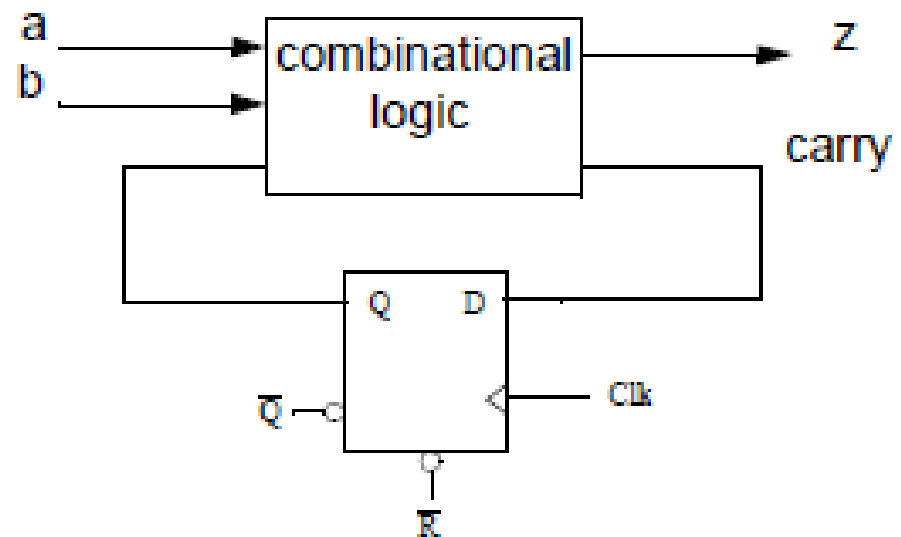
Machines de Mealy

- Représentation sur le graphe d'états
- État des sorties sur les arcs

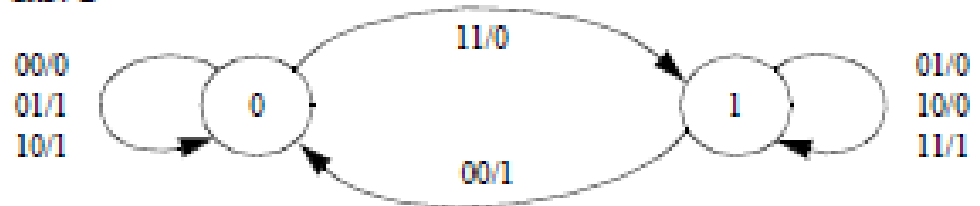


Machines de Mealy

- Additionneur serie 1 bit



ab/s



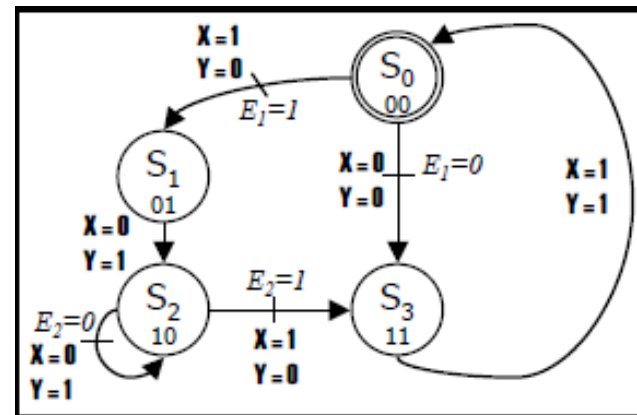
Machines de Mealy

- Équations de l'état futur

- Équations $Q_0^+ = \overline{Q_0} \cdot \overline{Q_1} + \overline{Q_0} \cdot E_2$ $Q_1^+ = Q_1 \oplus Q_0 + \overline{Q_1} \cdot E_1$

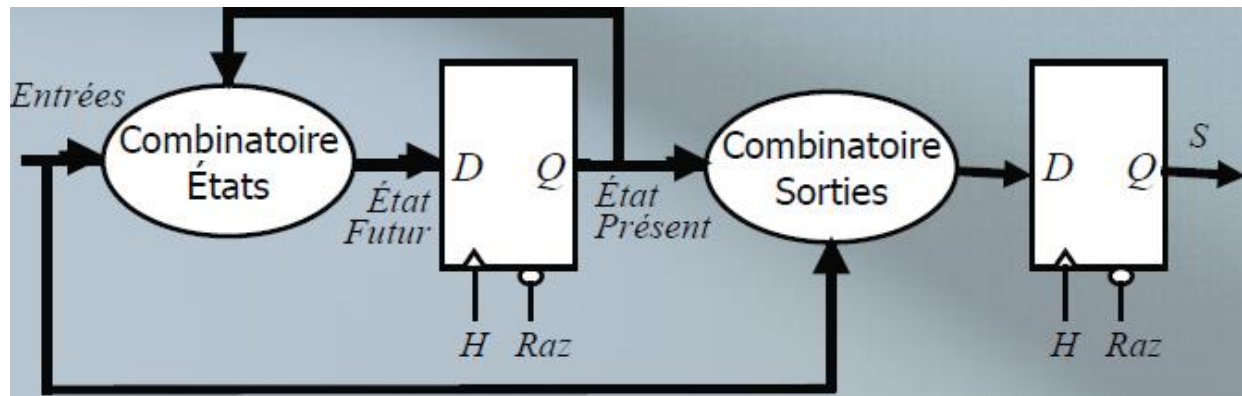
$$X = E_2 \cdot Q_1 + Q_0 \cdot Q_1 + E_1 \cdot \overline{Q_0} \cdot \overline{Q_1}$$

$$Y = Q_0 + Q_1 \cdot \overline{E_2}$$



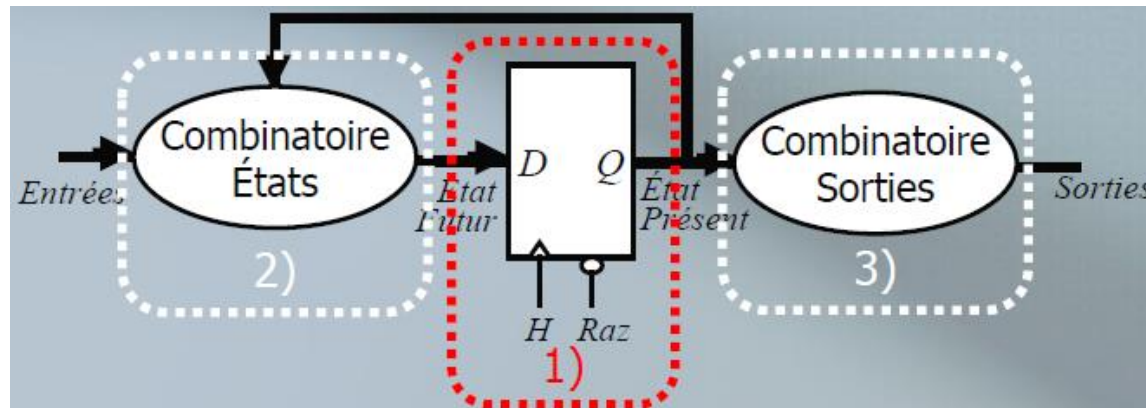
Machine de Mealy Synchrone

- Inconvénient Mealy:
 - Valeurs transitoires des sorties
 - Synchronisation
- Solution: Ajout d'un registre d'état



Machines à états - VHDL

- Description à l'aide de process
 - 1) Process décrivant le séquentiel (registre d'états)
 - 2) Process décrivant le combinatoire des états futurs
 - 3) Process décrivant le combinatoire des sorties
- 2) et 3) peuvent être mis dans un même process



Machine de Moore - VHDL

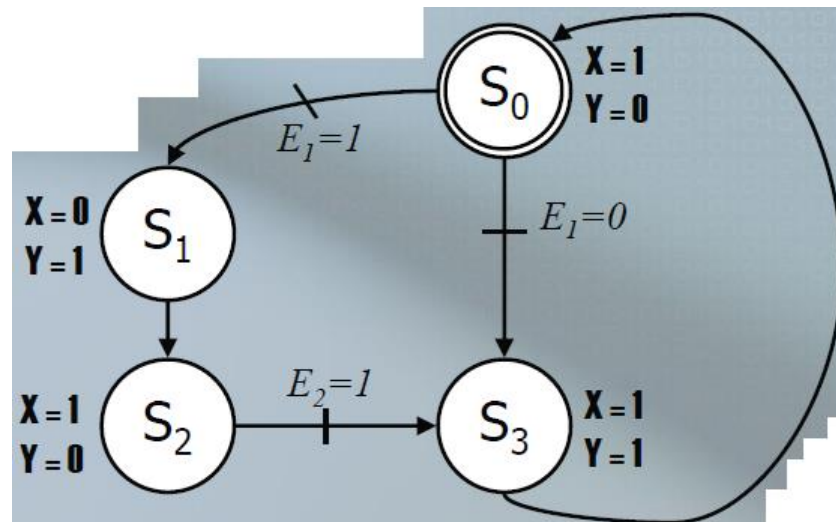
entity MAE **is**

port(h,raz: **in** std_logic;

e1,e2: **in** std_logic;

X,Y: **out** std_logic);

end MAE;



Machine de Moore - VHDL

architecture Moore3 **of** MAE **is**

-- Definition d'un type etat

type etat **is** (S0,S1,S2,S3); *-- Codage des états non spécifié. Établi par un
-- outil de synthèse en fonction de consignes*

signal EP, EF: etat;

begin

-- Process du Registre d'etats

process(h, raz)

begin

if raz='0' **then** EP <= S0;

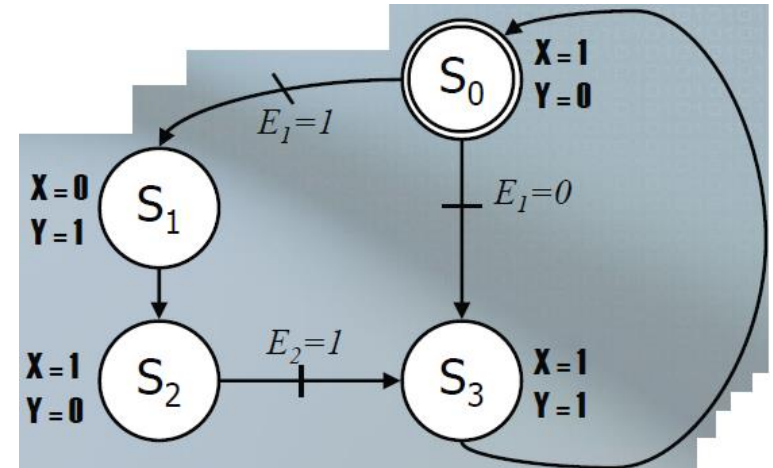
elsif rising_edge(h) **then** EP <= EF;

end if;

end process;

Machine de Moore - VHDL

- -- Combinatoire calcul etat suivant
- **process**(EP,e1,e2)
- **begin**
- **case** (EP) **is**
- **when** S0 => EF<=S3; **if** e1='1' **then** EF<=S1; **end if**;
- **when** S1 => EF<=S2;
- **when** S2 => EF<=S2; **if** e2='1' **then** EF<=S3; **end if**;
- **when** S3 => EF<=s0;
- **when others** => **null**;
- **end case**;
- **end process**;



L'état futur doit être
spécifié
Pour TOUTES les
combinaisons
des entrées et de l'état
présent

Machine de Moore - VHDL

-- Combinatoire des sorties

process(EP)

begin

case (EP) **is**

when S0 => X<='1';Y<='0';

when S1 => X<='0';Y<='1';

when S2 => X<='1';Y<='0';

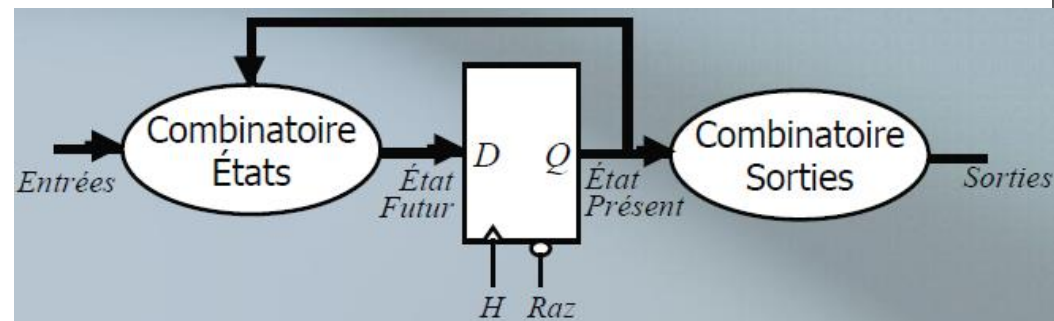
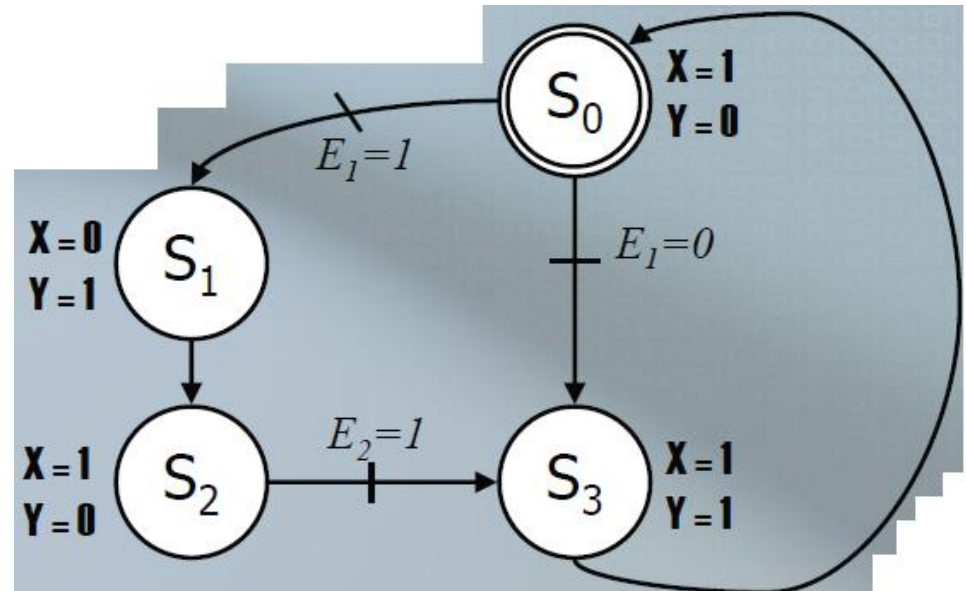
when S3 => X<='1';Y<='1';

when others => null;

end case;

end process;

end Moore3;



Machine de Moore - VHDL

-- Tout le combinatoire en un process

```
process(EP,e1,e2)
begin
X<='1';Y<='1';
case (EP) is
  when S0 => Y<='0';
  EF<=S3; if e1='1' then EF<=S1; end if;
  when S1 => X<='0';
  EF<=S2;
  when S2 => Y<='0';
  EF<=S2; if e2='1' then EF<=S3; end if;
  when S3 => EF<=s0;
  when others => null;
end case;
end process;
```

Machine de Mealy - VHDL

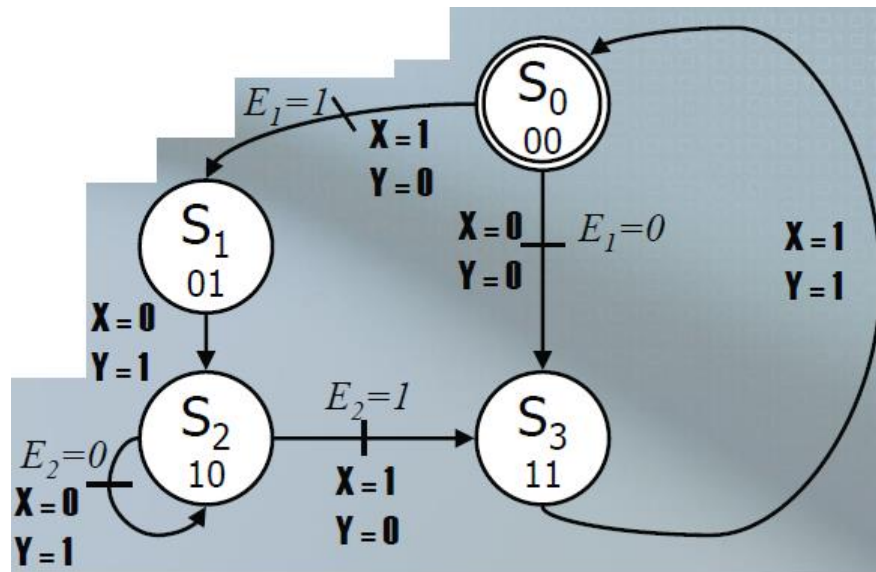
entity MAE **is**

port(h,raz: **in** std_logic;

e1,e2: **in** std_logic;

X,Y: **out** std_logic);

end MAE;



Machine de Mealy - VHDL

architecture Mealy of MAE is

-- Definition d'un type etat

type etat **is**(S0,S1,S2,S3);

signal EP, EF: etat;

begin

-- Process du Registre d'etats

process(h,raz)

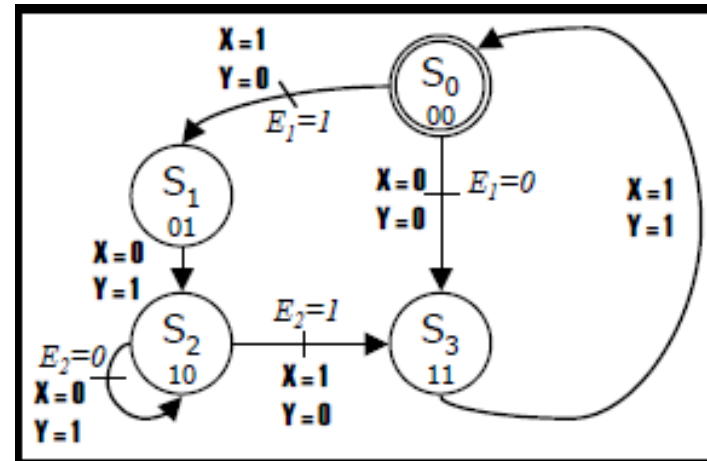
begin

if raz='0' **then** EP <= S0;

elsif rising_edge(h) **then** EP <= EF;

end if;

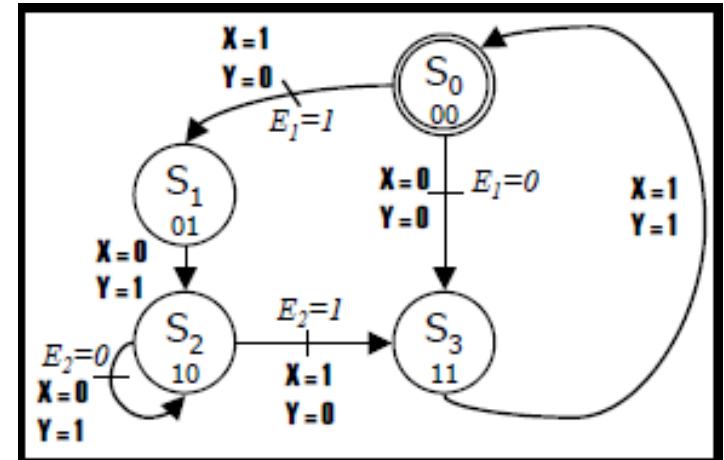
end process;



Machine de Mealy - VHDL

```

process(EP,e1,e2)
begin
X<='1';Y<='1';
case (EP) is
when S0 => if e1='1' then EF<=S1;Y<='0';
else EF<=S3; X<='0';Y<='0'; end if;
when S1 => X<='0'; EF<=S2;
when S2 => if e2='1' then EF<=S3;Y<='0';
else EF<=S2; X<='0'; end if;
when S3 => EF<=S0;
when others => null;
end case;
end process;
end Mealy;
    
```



Mealy Synchrone - VHDL

architecture Mealy2 **of** MAE **is**

-- Definition d'un type etat

type etat **is**(S0,S1,S2,S3);

signal EP, EF: etat;

-- Sortie du Combinatoire de sortie

signal C1,C2: **std_logic**;

begin

-- Process du Registre d'etats

process(h,raz)

begin

if raz='0' **then** EP <= S0; X<='0';Y<='0';

elsif **rising_edge**(h) **then** EP <= EF; X<=C1;Y<=C2;

end if;

end process;

Mealy Synchrone - VHDL

```
process(EP,e1,e2)
begin
  C1<='1'; C2<='1';
  case (EP) is
    when S0 => if e1='1' then EF<=S1; C2<='0';
    else EF<=S1; C1<='0'; C2<='0'; end if;
    when S1 => C1<='0'; EF<=S2;
    when S2 => if e2='1' then EF<=S3; C2<='0';
    else EF<=S2; C1<='0'; end if;
    when S3 => EF<=S0;
    when others => null;
  end case;
end process;
end Mealy2;
```

Exercice de consolidation MAE

- Exercice 1
- Réaliser le diagramme de Moore d'un système séquentiel dont l'unique sortie Y passe à 1 lorsque l'entrée X reste stable pendant 2 cycles d'horloges au minimum
Décrire le système en VHDL
- Proposer un diagramme de Mealy pour un distributeur de jus naturel. Pour obtenir une boisson, l'utilisateur doit entrer 500 Frs CFA, soit sous forme d'une pièce, soit sous forme de cinq pièces. La machine ne rend pas la monnaie. En cas de dépassement elle renvoie toutes pièces entrées;

Exercice de consolidation MAE (2)

- Exercice 2 MAE et chemin de données

Proposer un module Détecteur du double clic sur un bouton ; le module comprend une entrée (bp) et deux sorties s_clic et d_clic. S_clic est à 1 seulement lorsque une impulsion a été détectée sur le bouton dans l'intervalle de 300 ms. La sortie d_clic est à 1 seulement lorsque deux impulsions ont été détectées dans l'intervalle de 300 ms

Exercices de consolidation MAE et Datapath

Exercice 3 Module Temporisateur

Un système numérique permet de mesurer les durées, il comprend un registre compteur (cpt) 16 bits, un registre match 16 bits, une entrée de réinitialisation (init), une entrée d'activation (enable), une entrée mode (mode), une entrée de chargement du registre match, loadM, une entrée de chargement du registre compteur, loadC, une sortie MLI, et une sortie de débordement (Ovf), une entrée Din sur 16 bits et une sortie Qout sur 16 bits

La sortie ovf passe à 1 chaque fois que le compteur déborde (passe de 0xFFFF à 0x00000)

La sortie MLI passe à 1 au début du comptage et revient à 0 lorsque la valeur du compteur est supérieure à la valeur du registre match

L'entrée LoadM (resp LoadC) permet de charger le registre compteur (resp le registre match) lorsqu'elle vaut 1.

L'entrée init ramène compteur à zero et match à 0x7FFF lorsqu'elle vaut 1

La valeur du registre cpt est incrémentée lorsque enable vaut 1,

Si l'entrée mode vaut 1, le compteur recommence automatiquement, sinon il s'arrête à la valeur maxi 0xFFFF et il faut le réinitialiser (init=1) pour qu'il recommence le comptage.

1. Modéliser ce module temporisateur sous forme d'un contrôleur et d'un chemin de données
2. Décrire ce module temporisateur à l'aide du VHDL

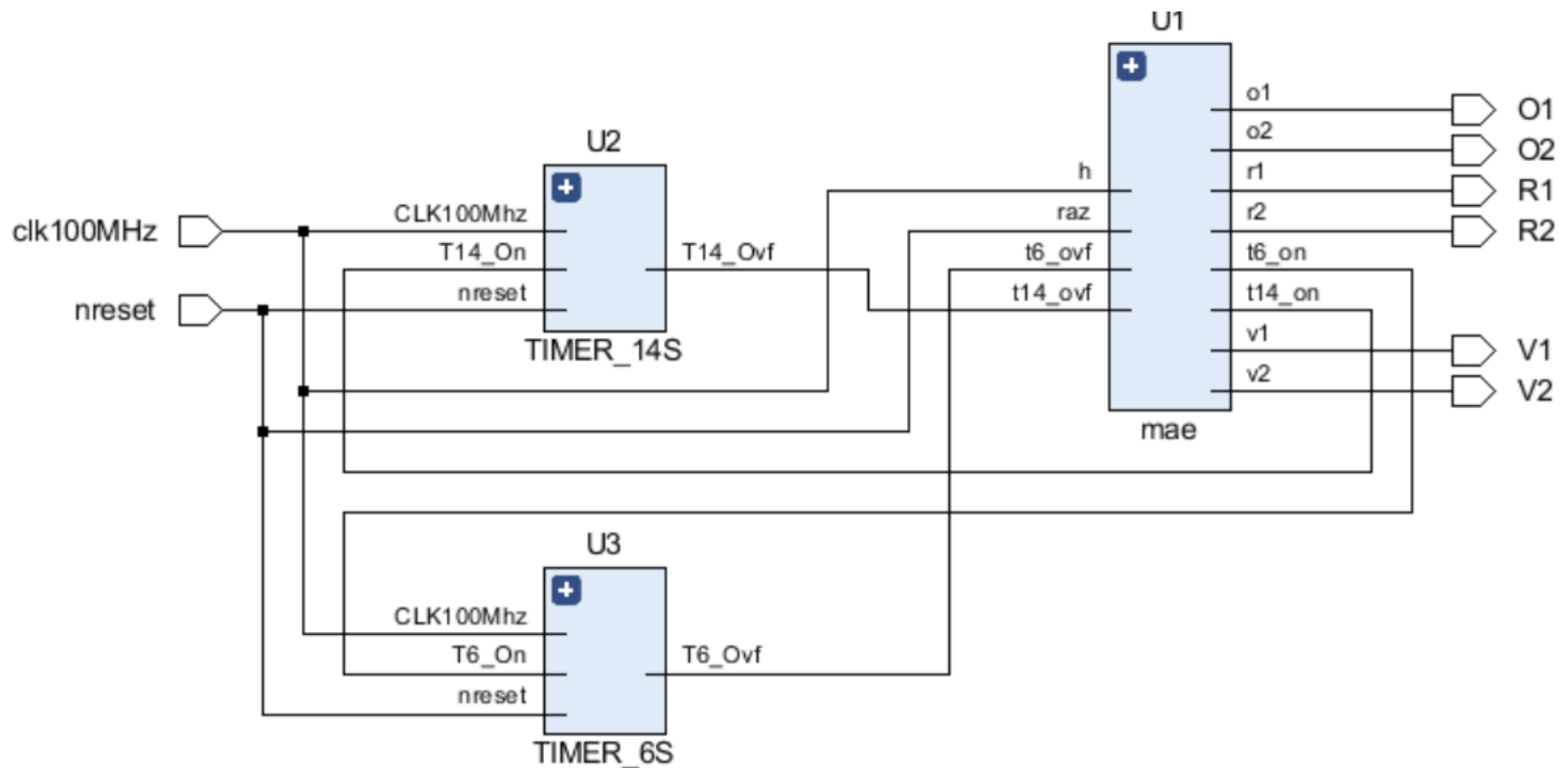
Exercice de consolidation : les feux tricolores

- Au lieu dit Mobil bonakouanmouang, le feu tricolore fonctionne de la manière suivante :
 - À l'instant initial, la voie 1 est au vert et la voie 2 est au rouge
 - 14 s plus tard, la voie 1 est à l'orange tandis que la voie 2 reste au rouge
 - 6 s plus tard la voie 1 passe au rouge et la voie 2 passe au vert
 - Le même cycle se produit pour la voie 2.
 - Pour chaque voie, le rouge dure 20 s, l'orange 6s et le vert 14 s
 - Les modules de temporisation T_6s (resp T_14s) comprennent chacun une entrée pour la mise en route et une sortie qui passe à un lorsque le délai est écoulé
 - TAF :
 - Proposer un processeur dédié pour la gestion du feu tricolore sachant que l'horloge du CI FPGA est fixé à 100 MHz
 - Réaliser l'implémentation en VHDL de la MAE et du chemin de données

Exercice de consolidation : les feux tricolores

- TAF :

- Proposer un processeur dédié pour la gestion du feu tricolore sachant que l'horloge du CI FPGA est fixé à 100 MHz
- Réaliser l'implémentation en VHDL structurel de la MAE et des différentes temporisations



Cas d'étude Distribution des Jouets par GPS

- Module MAE

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity CTRL_MAE is

Port (H,raz : in STD_LOGIC;

LOAD_GPS : out STD_LOGIC;

START_GPS : out STD_LOGIC;

OK : in STD_LOGIC;

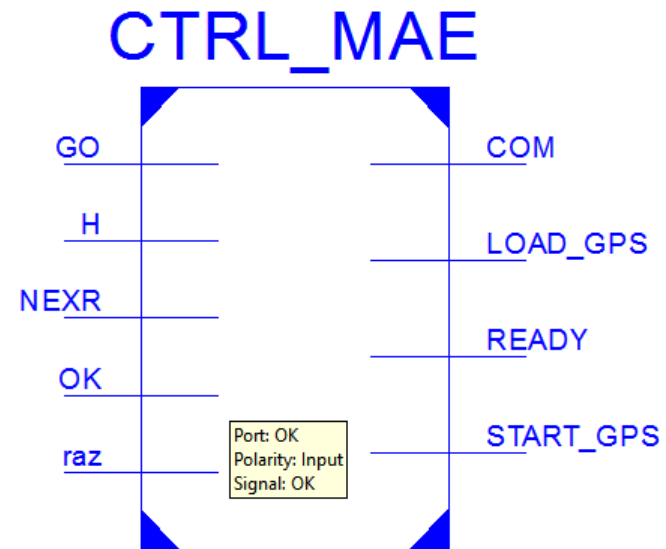
READY : out STD_LOGIC;

COM : out STD_LOGIC;

NEXR : in STD_LOGIC;

GO : in STD_LOGIC);

end CTRL_MAE;



```

architecture Moore3 of CTRL_MAE is
-- Definition d'un type etat
type etat is (Debut,INC_ADR,COORD_PROCHE,ARRIVEE,
Attente_pos, Charge_GPS,Guidage_GPS); -- Codage des états non
spécifié.
signal EP, EF: etat;
begin
-- Process sequentiel synchrone du Registre d'etats
SYNC_PROC: process(h, raz)
begin
if raz='0' then EP <= DEBUT;
elsif rising_edge(h) then
EP <= EF;
end if;
end process;

```

```
-- process Combinatoire de calcul etat futur
EF_PROC:process(EP,NEXR,GO,OK)
begin
```

```
EF<=EP; --valeur par défaut
case (EP) is
when Debut => if NEXR='1' then
                EF<=INC_ADR;
            end if;
when INC_ADR => EF<= ATTENTE_POS;
when ATTENTE_POS
=>EF<=COORD_PROCHE;
when COORD_PROCHE => IF OK='1' and
GO='1' then
```

```
                EF<=CHARGE_GPS;
            elsif OK='0' or NEXR='1' then
```

```
                EF<=INC_ADR;
            end if;
when Charge_GPS => IF GO='1' then
                EF<=Guidage_GPS;
            elsif NEXR='1' then
                EF<=INC_ADR;
            end IF;
when Guidage_GPS => IF OK ='1' then
                EF<=ARRIVEE;
            end IF;

when ARRIVEE => IF NEXR='1' then
                EF<=INC_ADR;
            end IF;

end case;
end process;
```

```

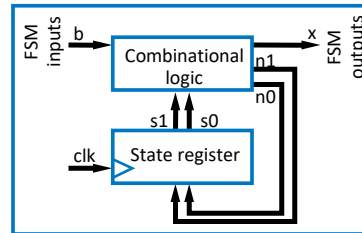
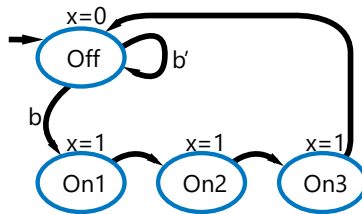
--process combinatoire d'affectation des sortie
OUT_PROC:process(EP)
Begin
-valeurs par défaut
READY<='0'; COM<='0'; LOAD_GPS<='0'; START_GPS<='0';
case (EP) is
when Debut =>COM<='0';
when INC_ADR =>COM<='1';
when COORD_PROCHE =>READY<='1';
when Charge_GPS => READY<='1';
when Guidage_GPS => START_GPS<='1'; -- fait un reset de l'ADR
when ARRIVEE =>READY<='1';
when ATTENTE_POS =>

end case;
end process;
end Moore3;

```

Description d'un contrôleur en VHDL

Inputs: b; Outputs: x



FSM
outputs

- FSM behavior captured using architecture with 2 processes
 - First process models state register
 - Asynchronous reset sets state to "S_Off"
 - Rising clock edge sets currentstate to nextstate
 - Second process models combinational logic
 - Sensitive to currentstate and FSM inputs
 - Sets FSM outputs based on currentstate
 - Sets nextstate based on currentstate and present FSM input values
 - Note declaration of new type, statetype

```

library ieee;
use ieee.std_logic_1164.all

entity LaserTimer is
  port (b: in std_logic;
        x: out std_logic;
        clk, rst: in std_logic
        );
end LaserTimer;

architecture behavior of LaserTimer is
  type statetype is
    (S_Off, S_On1, S_On2, S_On3);
  signal currentstate, nextstate:
    statetype;
begin
  statereg: process(clk, rst)
  begin
    if (rst='1') then -- initial state
      currentstate <= S_Off;
    elsif (clk='1' and clk'event) then
      currentstate <= nextstate;
    end if;
  end process;

  comblogic: process (currentstate, b)
  begin
    case currentstate is
      when S_Off =>
        x <= '0'; -- laser off
        if (b='0') then
          nextstate <= S_Off;
        else
          nextstate <= S_On1;
        end if;
      when S_On1 =>
        x <= '1'; -- laser on
        nextstate <= S_On2;
      when S_On2 =>
        x <= '1'; -- laser still on
        nextstate <= S_On3;
      when S_On3 =>
        x <= '1'; -- laser still on
        nextstate <= S_Off;
    end case;
  end process;
end behavior;
  
```

High-Level State Machine of the Laser-Based Distance Measurer in VHDL

- Architecture similar to FSM, but single process
- Asynch reset forces to state S0
- For each rising clock
 - Perform state's computation
 - Prepare to go to next state based on state and inputs

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;

entity LaserDistMeasurer is
    port ( clk, rst: in std_logic;
          B, S: in std_logic;
          L: out std_logic;
          D: out unsigned(15 downto 0) );
end LaserDistMeasurer;

architecture behavior of LaserDistMeasurer is
    type statetype is (S0, S1, S2, S3, S4);

    signal state : statetype;
    signal Dctr : unsigned(15 downto 0);

    constant U_ZERO :
        unsigned (15 downto 0) := "0000000000000000";
    constant U_ONE : unsigned(0 downto 0) := "1";

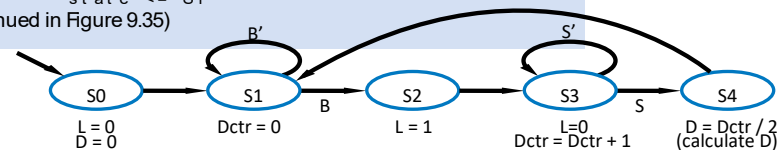
begin
    statemachine: process(clk, rst)
    begin
        if (rst='1') then
            L <= '0';
            D <= U_ZERO;
            Dctr <= U_ZERO;
            state <= S0;           -- initial state
        elsif (clk='1' and clk'event) then
            case state is
                when S0 =>
                    L <= '0';           -- laser off
                    D <= U_ZERO;       -- clear D
                    state <= S1;
            end case;
        end if;
    end process;
end behavior;
    
```

(continued in Figure 9.35)

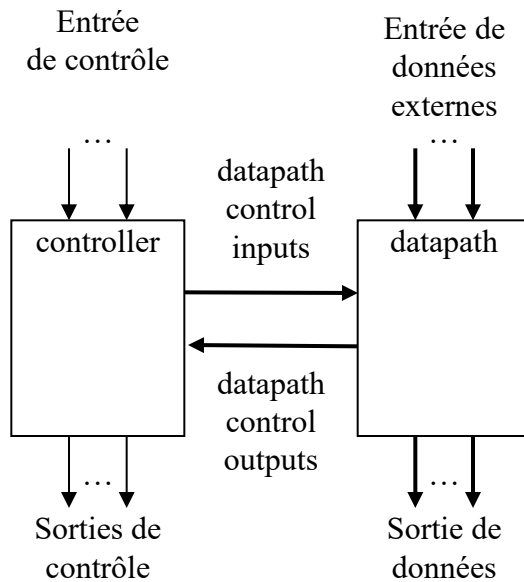
(continued from Figure 9.34)

```

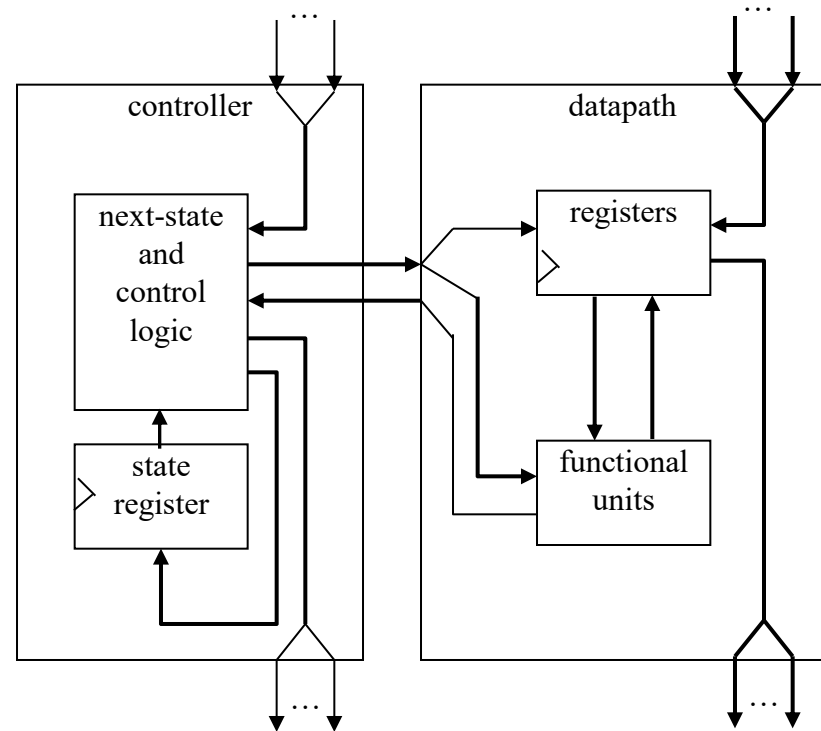
        when S1 =>
            Dctr <= U_ZERO;           -- reset count
            if (B='1') then
                state <= S2;
            else
                state <= S1;
            end if;
        when S2 =>
            L <= '1';                 -- laser on
            state <= S3;
        when S3 =>
            L <= '0';                 -- laser off
            Dctr <= Dctr + 1;
            if (S='1') then
                state <= S4;
            else
                state <= S3;
            end if;
        when S4 =>
            D <= SHR(Dctr, U_ONE); -- calculate D
            state <= S1;
        end case;
    end if;
end process;
end behavior;
    
```



Modèle de base de processeur dédié à une application (Contrôleur + Chemin de données)



Contrôleur et chemin de données

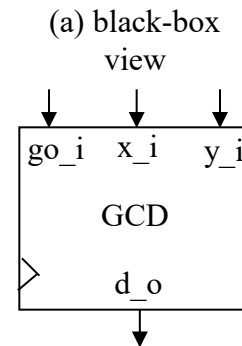


Une vue à l'intérieur du contrôleur et du chemin de données

conception au niveau algorithmique

Exemple: le plus grand diviseur commun

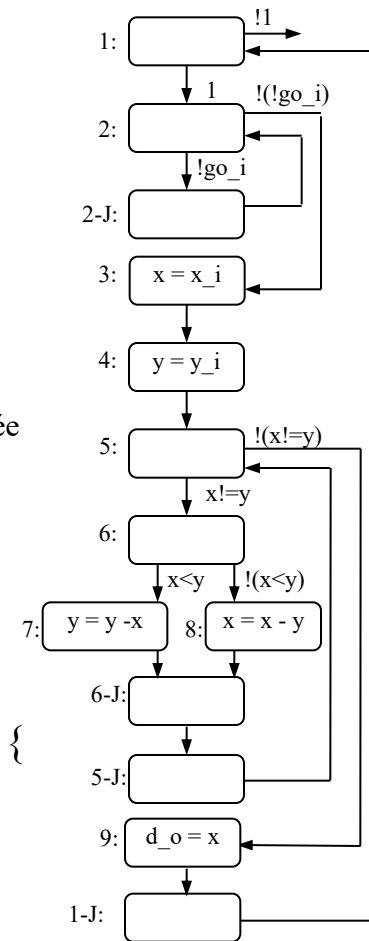
- Partir d'un algorithme de conception
- Convertir l'algorithme en machine à états "complexe"
 - Connu sous le nom de FSMD: machine à états finis avec datapath
 - Peut utiliser des modèles pour effectuer une telle conversion



(b) fonctionnalité désirée

```

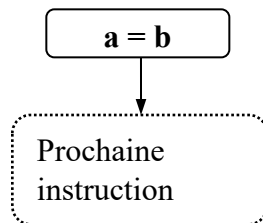
0: int x, y;
1: while (1) {
2:   while (!go_i);
3:   x = x_i;
4:   y = y_i;
5:   while (x != y) {
6:     if (x < y)
7:       y = y - x;
8:     else
7:       x = x - y;
9:   }
9:   d_o = x;
}
    
```



Templates de conversion de l'algorithme en diagramme d'état complexe

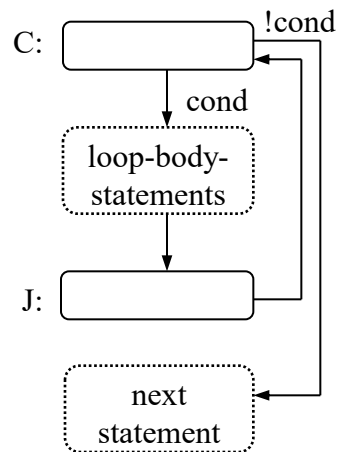
Affectation

a = b
Prochaine
instruction



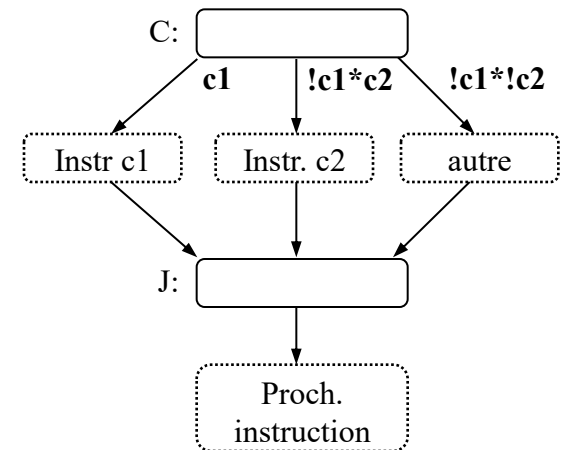
Boucle

while (cond) {
 loop-body-
 statements
}
next statement



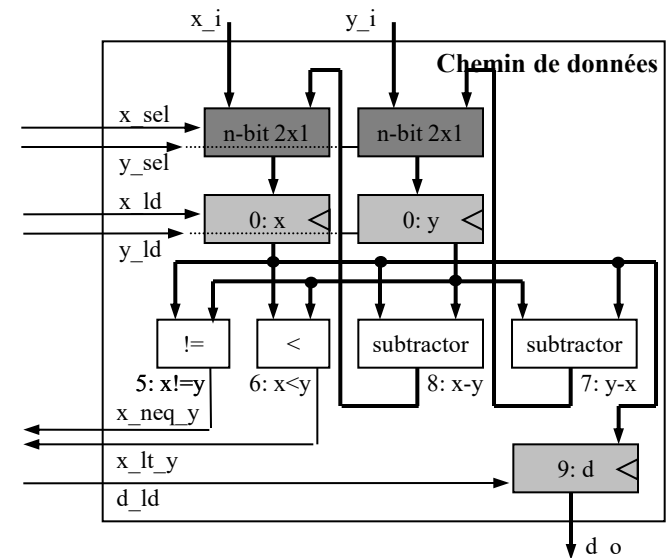
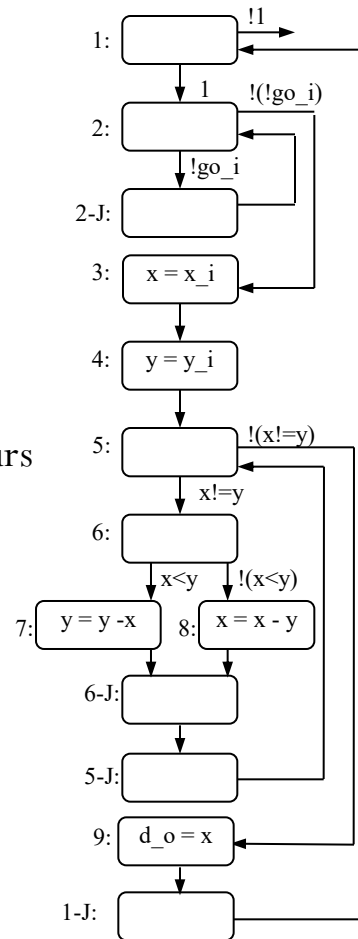
Instruction conditionnel

if (c1)
 instr.c1
else if c2
 instr.c2
else
 autre instr
Prochaine
instruction

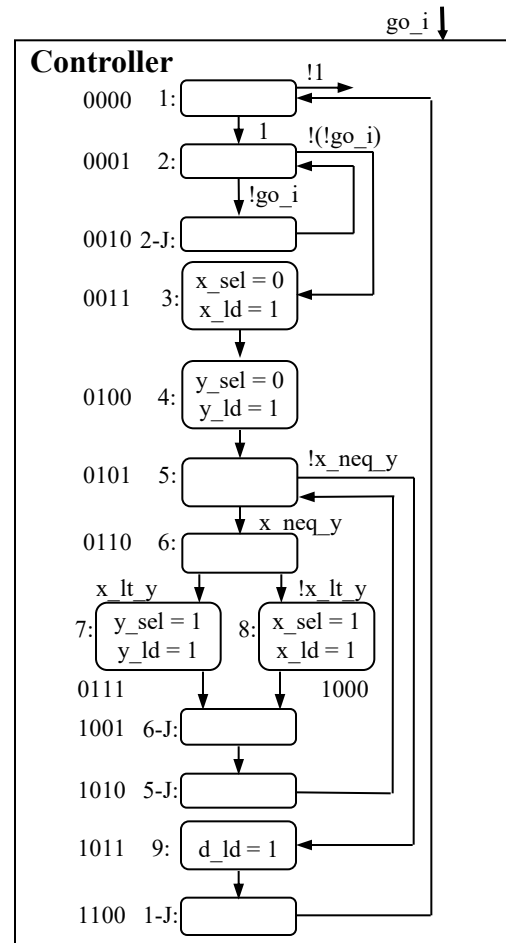
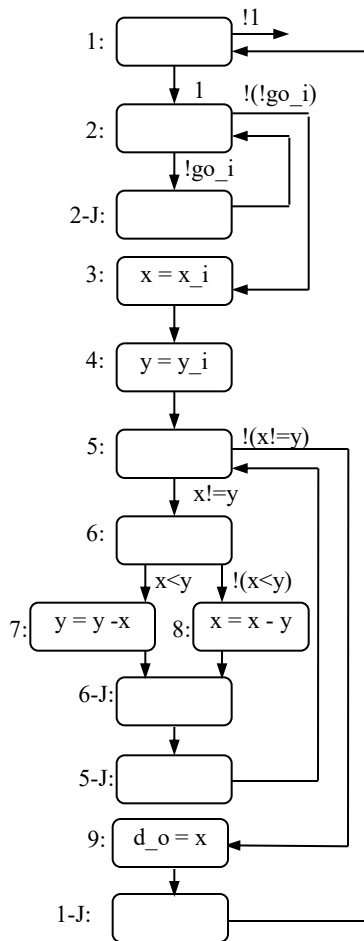


Principe de création d'un chemin de données

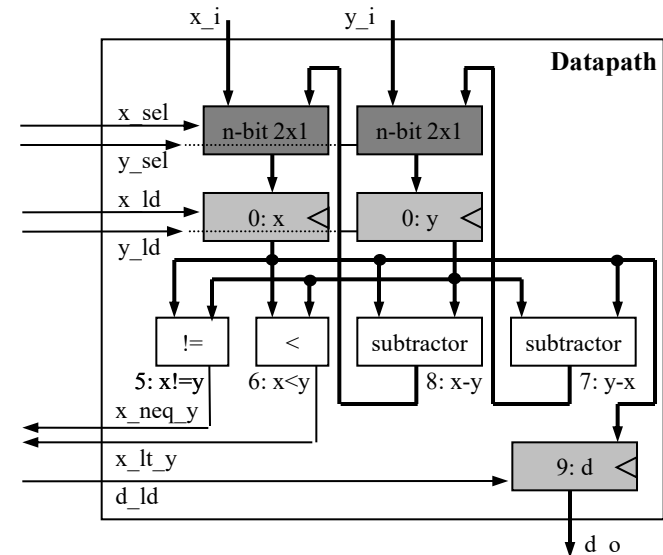
- Créer un registre pour toute variable déclarée
- Créer une unité fonctionnelle pour chaque opération arithmétique
- Connectez les ports, les registres et les unités fonctionnelles
 - Basé sur les lectures et écritures
 - Utiliser des multiplexeurs pour plusieurs sources
- Créer un identifiant unique
 - Pour chaque composant du chemin de données contrôler les entrée et sorties



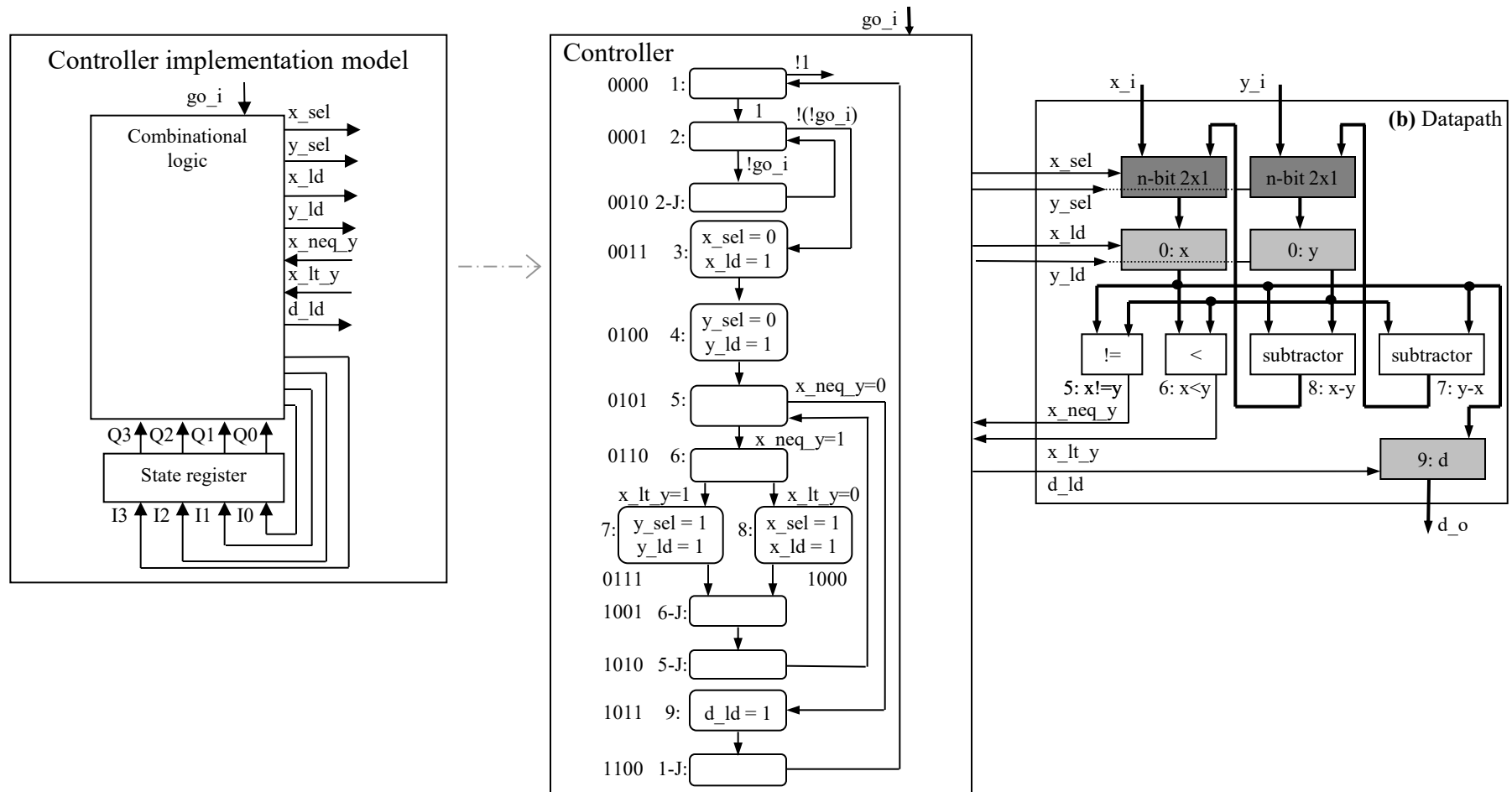
Création du contrôleur ou FSM



- Même structure que FSMD
- Remplacer les actions / conditions complexes par des configurations de datapath

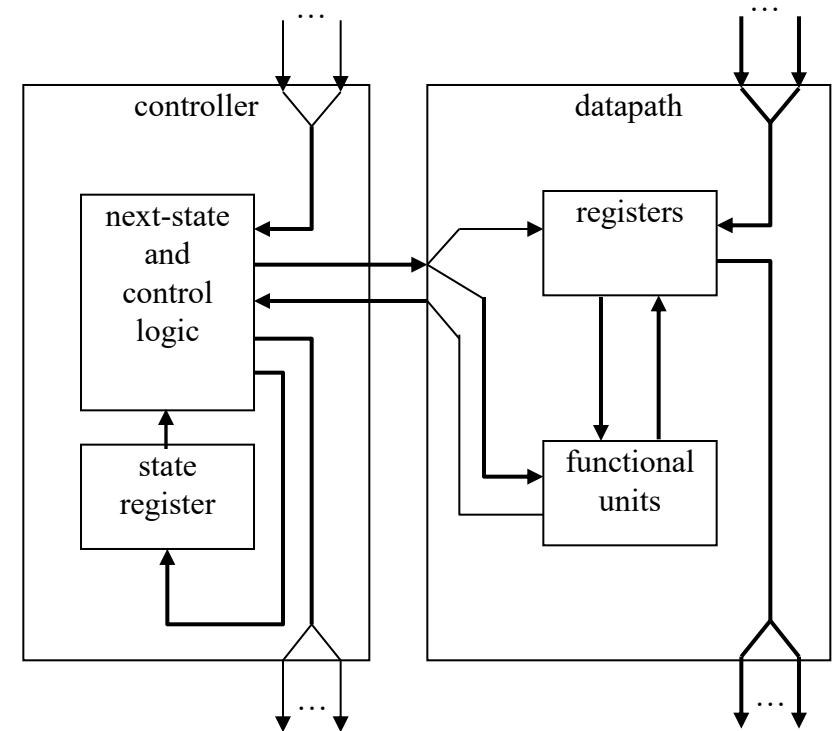


Fractionnement FSMD en un contrôleur et un chemin de données



Terminer la conception de processeur personnalisé GCD

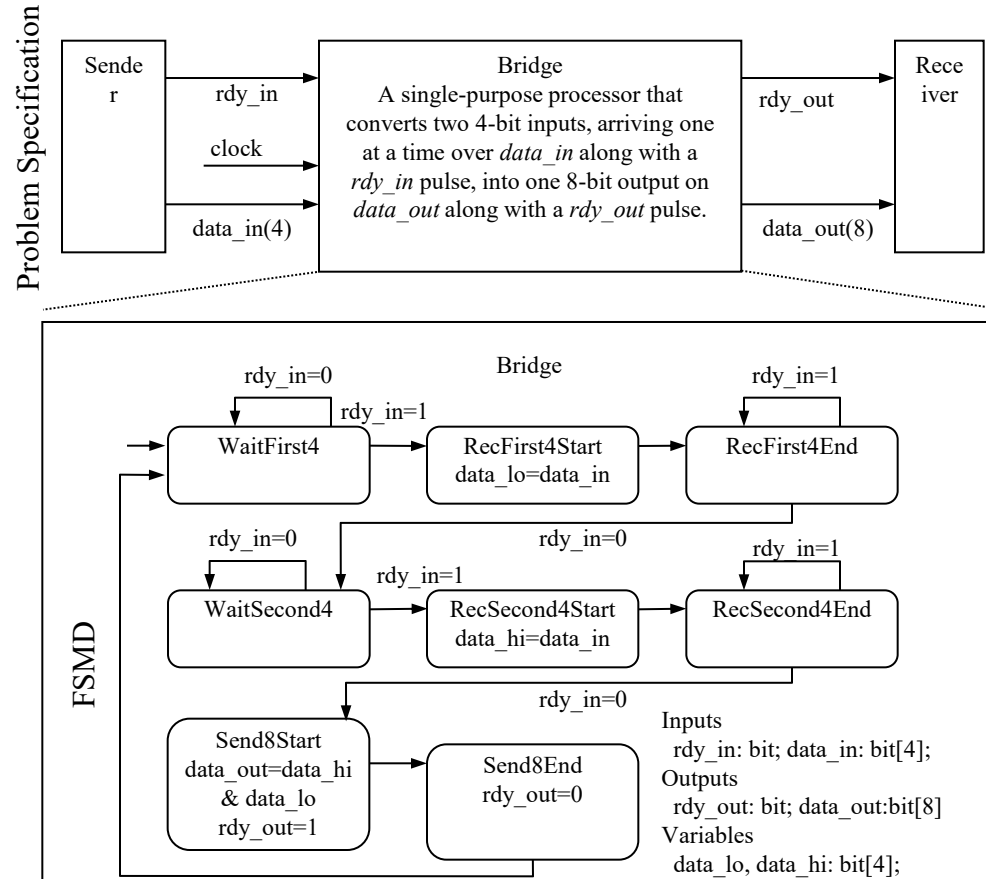
- Nous avons fini le chemin de données
- Nous avons une table d'état pour la prochaine logique d'état et de contrôle
 - Tout ce qui reste est la conception logique combinatoire
- Ce n'est pas un design optimisé, mais nous voyons les étapes de base



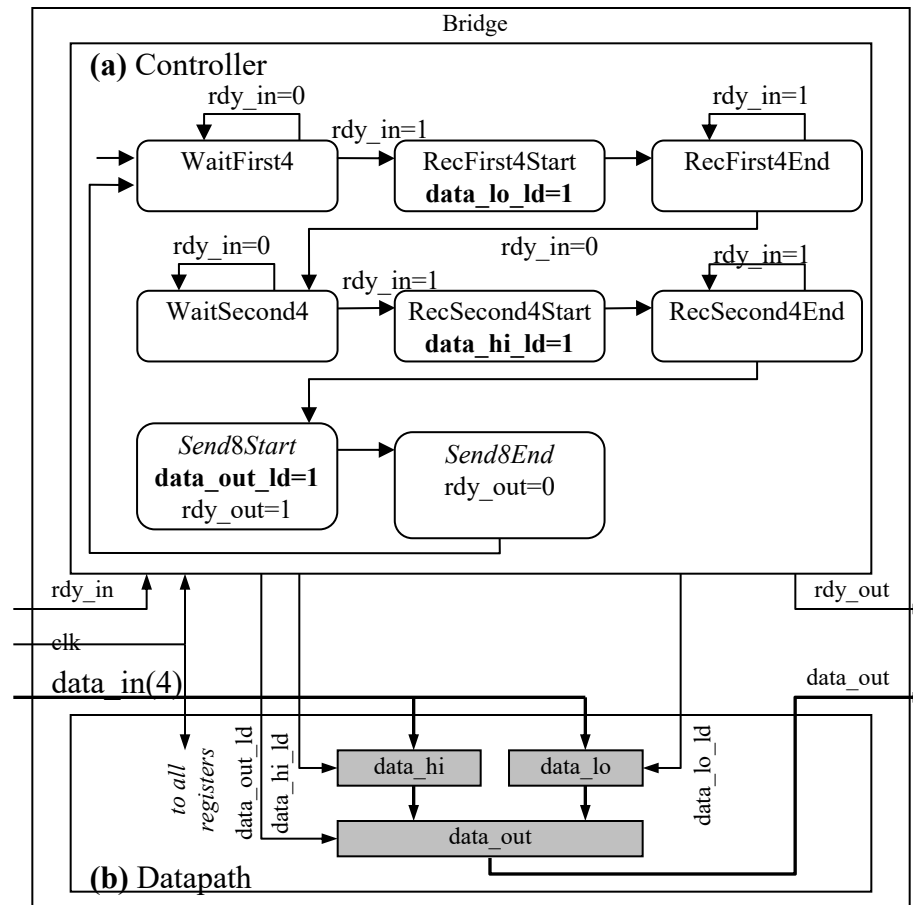
a view inside the controller and datapath

Conception de processeur personnalisé au niveau RTL

- Nous commençons souvent par une machine d'état
 - Plutôt que par l'algorithme
 - temps de cycle trop souvent au cœur de la fonctionnalité
- Exemple
 - Pont de bus qui convertit le bus 4 bits en bus 8 bits
 - Commencez avec une FSMD
 - Appelé Register Transfer Logic (RTL)
- Exercice: complétez le design



RT-level custom single-purpose processor design (cont')



Optimisation du programme original

- Analyser les attributs du programme et rechercher les domaines d'amélioration possible
 - Nombre de calculs
 - Taille de variable
 - Complexité du temps et de l'espace
 - Opérations utilisées
 - la multiplication et la division très cher

Optimisation du programme original (suite)

original program

```
0: int x, y;
1: while (1) {
2:   while (!go_i);
3:   x = x_i;
4:   y = y_i;
5:   while (x != y) {
6:     if (x < y)
7:       y = y - x;
8:     else
9:       x = x - y;
10:  }
11:  d_o = x;
12: }
```

Remplacez les opérations de soustraction par une opération modulo afin d'accélérer le programme

optimized program

```
0: int x, y, r;
1: while (1) {
2:   while (!go_i);
3:   // x must be the larger number
4:   if (x_i >= y_i) {
5:     x=x_i;
6:     y=y_i;
7:   }
8:   else {
9:     x=y_i;
10:    y=x_i;
11:  }
12:  while (y != 0) {
13:    r = x % y;
14:    x = y;
15:    y = r;
16:  }
17:  d_o = x;
18: }
```

GCD(42, 8) - 9 iterations to complete the loop

x and y values evaluated as follows : (42, 8), (43, 8), (26,8), (18,8), (10, 8), (2,8), (2,6), (2,4), (2,2).

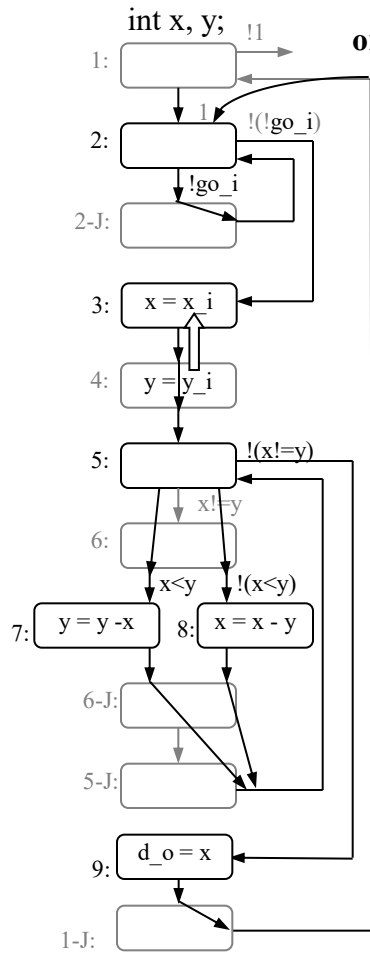
GCD(42,8) - 3 iterations to complete the loop

x and y values evaluated as follows: (42, 8), (8,2), (2,0)

Optimisation de la FSMD

- Domaines d'amélioration possibles
 - Fusionner des états
 - Les états avec des constantes sur les transitions peuvent être éliminés, la transition prise est déjà connue
 - Les états avec des opérations indépendantes peuvent être fusionnés
 - États distincts
 - Les états nécessitant des opérations complexes ($a * b * c * d$) peuvent être divisés en états plus petits pour réduire la taille du matériel
- Planification

Optimisation de la FSMD (suite)



eliminate state 1 – transitions have constant values

merge state 2 and state 2J – no loop operation in between them

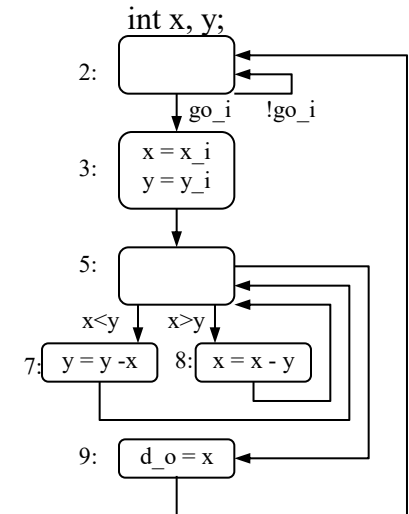
merge state 3 and state 4 – assignment operations are independent of one another

merge state 5 and state 6 – transitions from state 6 can be done in state 5

eliminate state 5J and 6J – transitions from each state can be done from state 7 and state 8, respectively

eliminate state 1-J – transition from state 1-J can be done directly from state 9

optimized FSMD



Optimisation du chemin de données

- Partage des unités fonctionnelles
 - Le mapping un à un, comme précédemment n'est pas nécessaire
 - Si la même opération se produit dans différents états, ils peuvent partager une seule unité fonctionnelle
- Unités multifonctions
 - Les ALU prennent en charge diverses opérations, elles peuvent être partagées entre les opérations qui se déroulent dans différents états

Optimisation des processeurs à usage unique

- L'optimisation est la tâche de rendre les métriques de conception les meilleures possibles
- Opportunités d'optimisation
 - Programme original
 - FSMD
 - datapath
 - FSM

Optimisation de la FSM

- Codage d'état
 - Tâche d'attribuer un modèle de bits unique à chaque état dans une FSM
 - La taille du registre d'état et la logique combinatoire varient
 - Peut être traité comme un problème de commande
- Minimisation de l'état
 - Tâche consistant à fusionner des états équivalents en un seul état
 - États équivalents si, pour toutes les combinaisons d'entrées possibles, les deux états génèrent les mêmes sorties et les mêmes transitions vers le même état

Résumé

- Les processeurs dédiés à une application
 - Utilisent des techniques de conception simples
 - Peuvent être réalisés pour exécuter des algorithmes
 - Généralement, conçus au niveau RTL en commençant par une FSMD
 - Les outils de CAO sont d'une grande aide pour leur implémentation